



<https://haoailab.com/cse291-s26/>

CSE/DSC 291: Deep Learning Systems Spring 2026

LLM and case studies

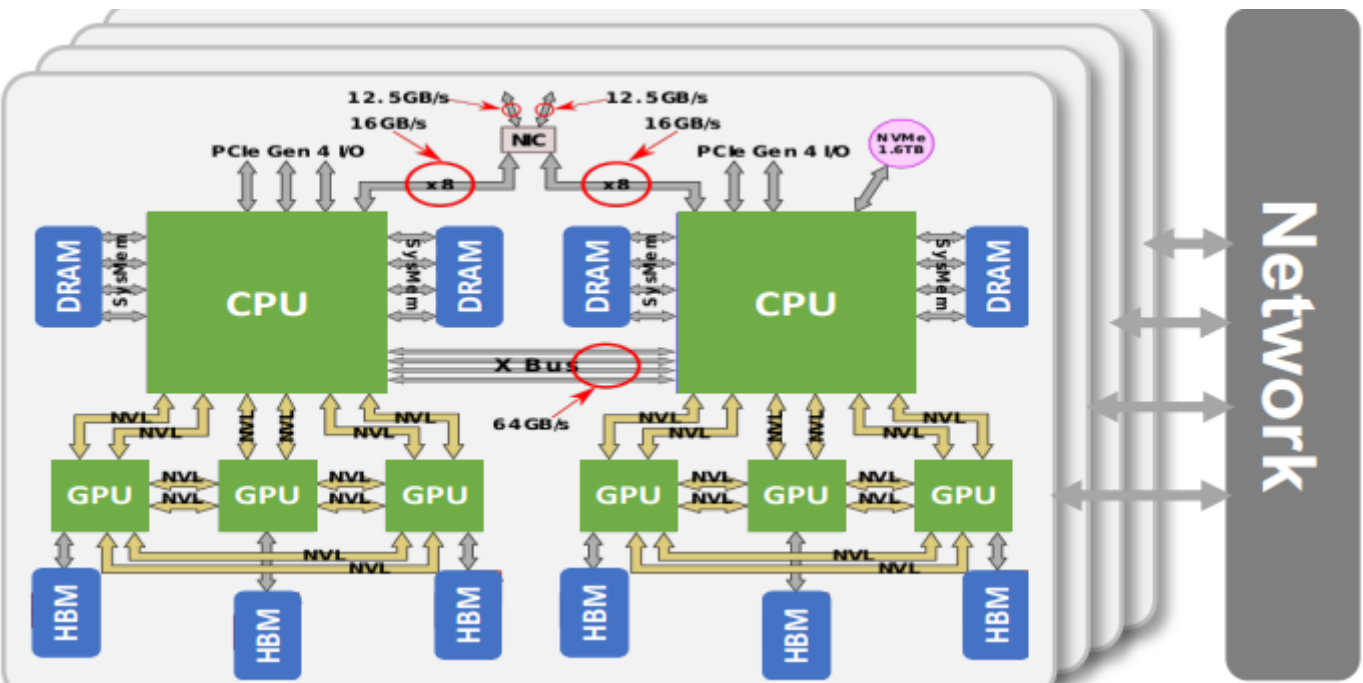
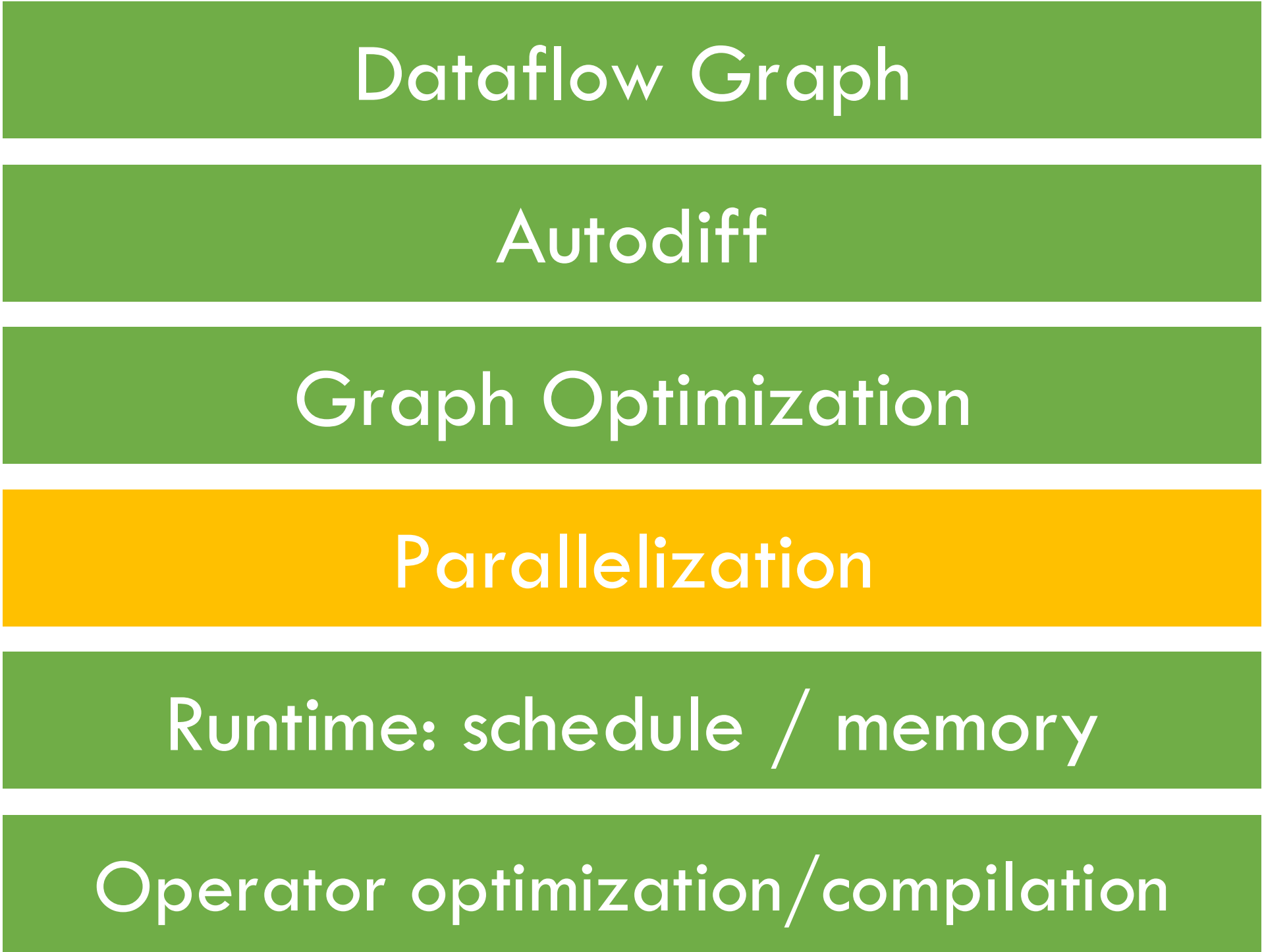
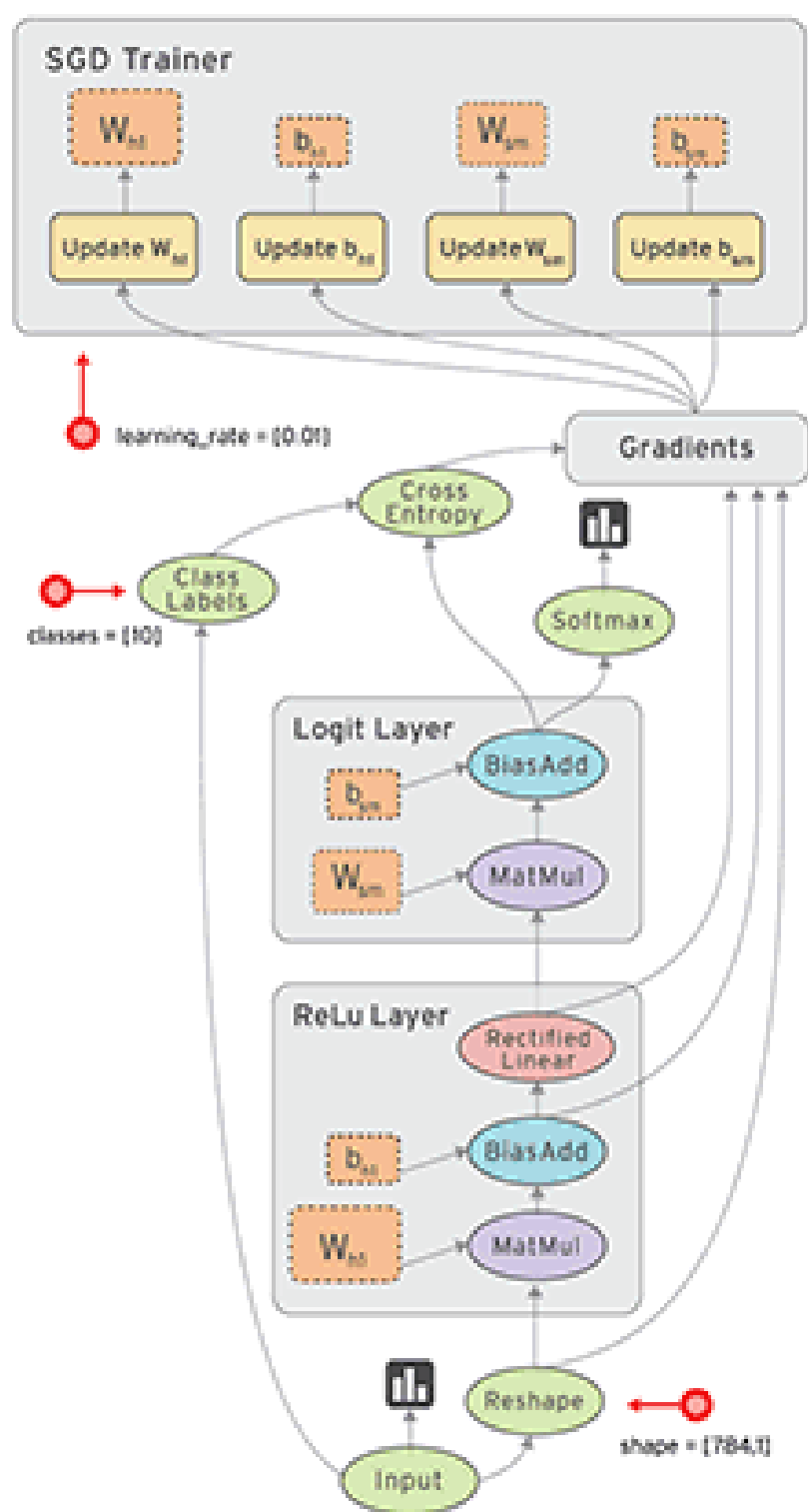
Optimizations and **Parallelization**

Basics

Logistics

- End of quarter evaluation (SET)
 - 52% in CSE
 - 31% in DSC
 - Due: end of this week
- Please fill in the evaluation
 - Again: if 80% of you filled the eval, you will get another 1%.
- See you next week in exam.

Big Picture



Parallelization

- **Why Parallelization: Technology Trend**
- ML Parallelism Overview
- Collective Communication Overview
- Inter and intra-op parallelism

Dataflow Graph

Autodiff

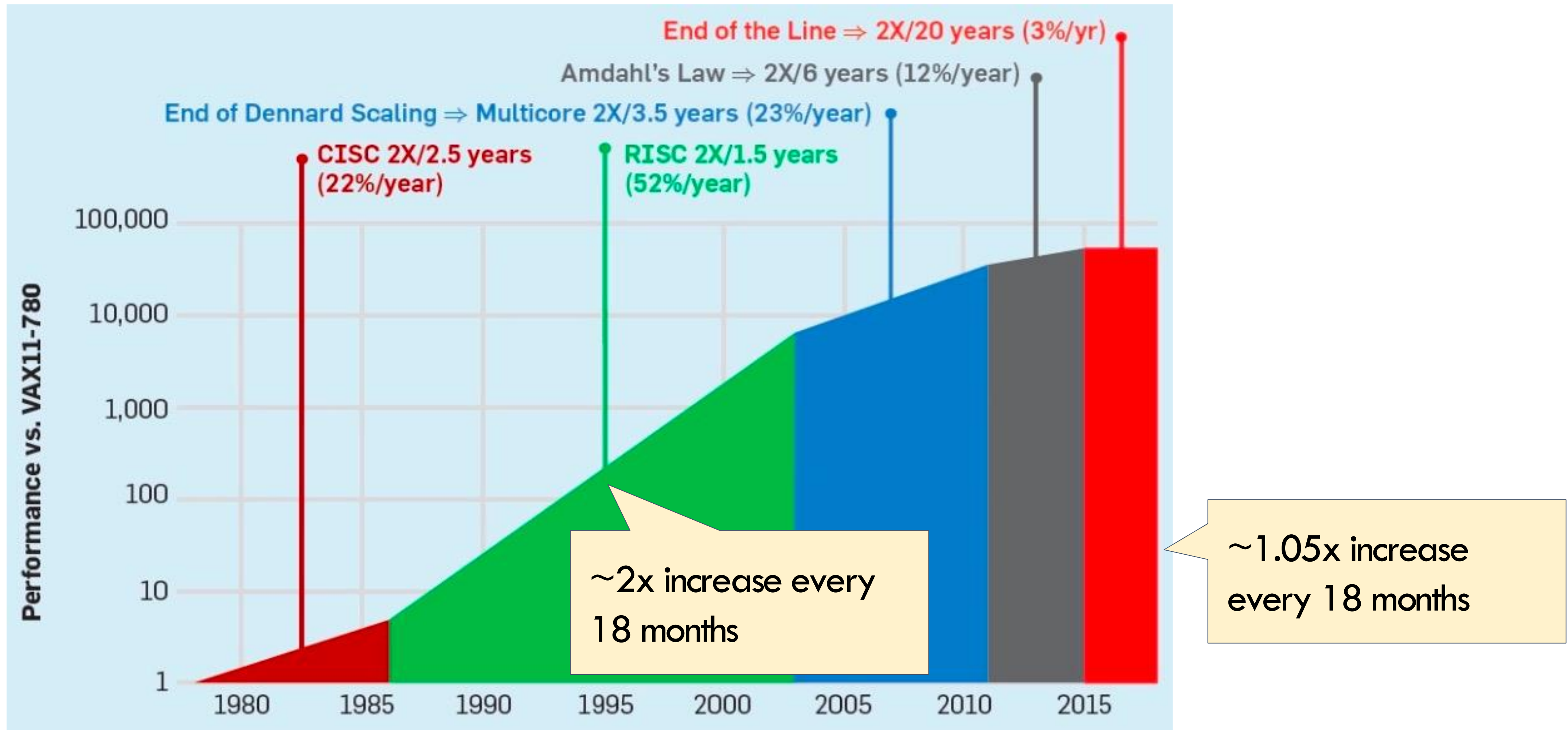
Graph Optimization

Parallelization

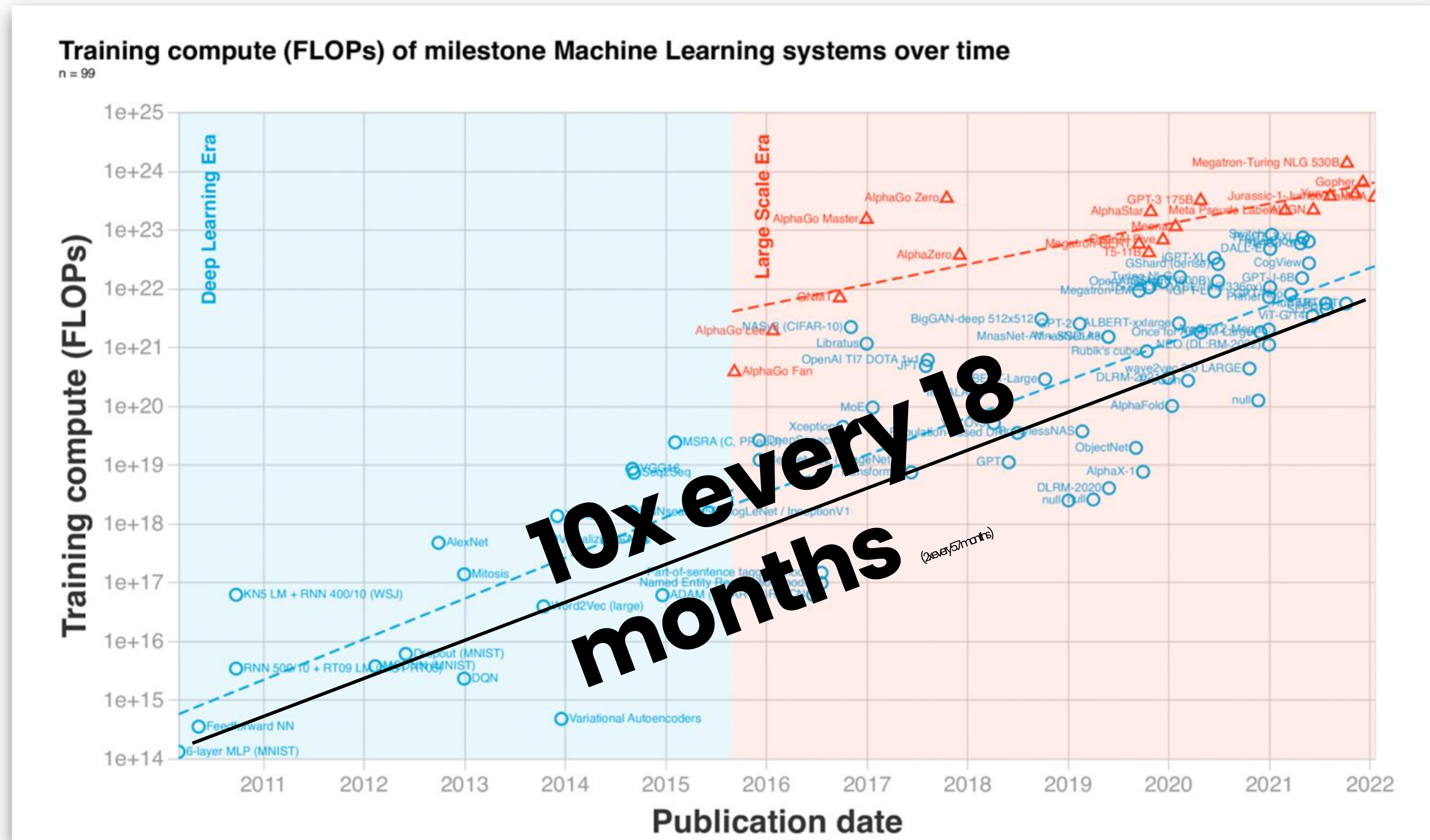
Runtime

Operator

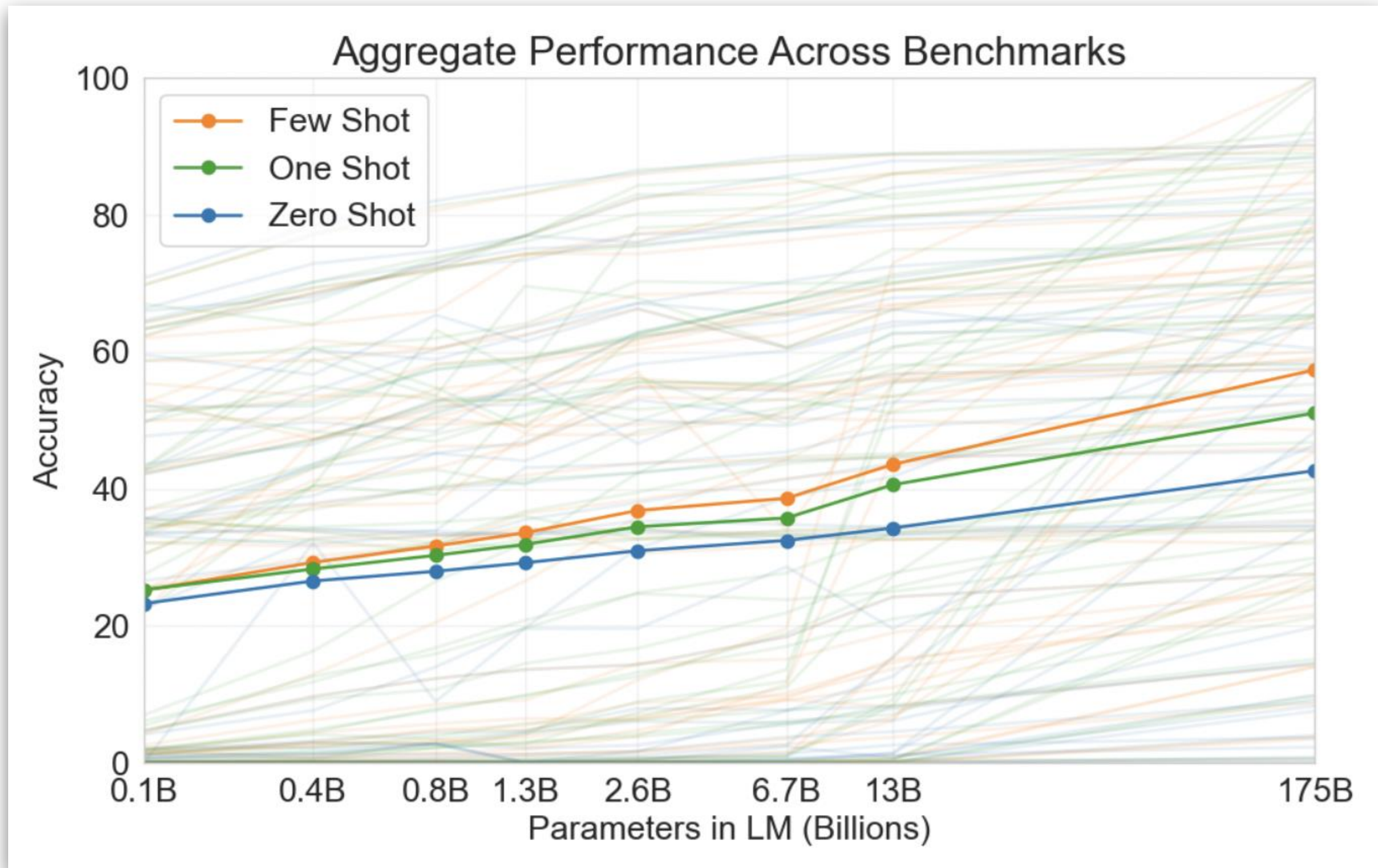
Moore's Law coming to an end



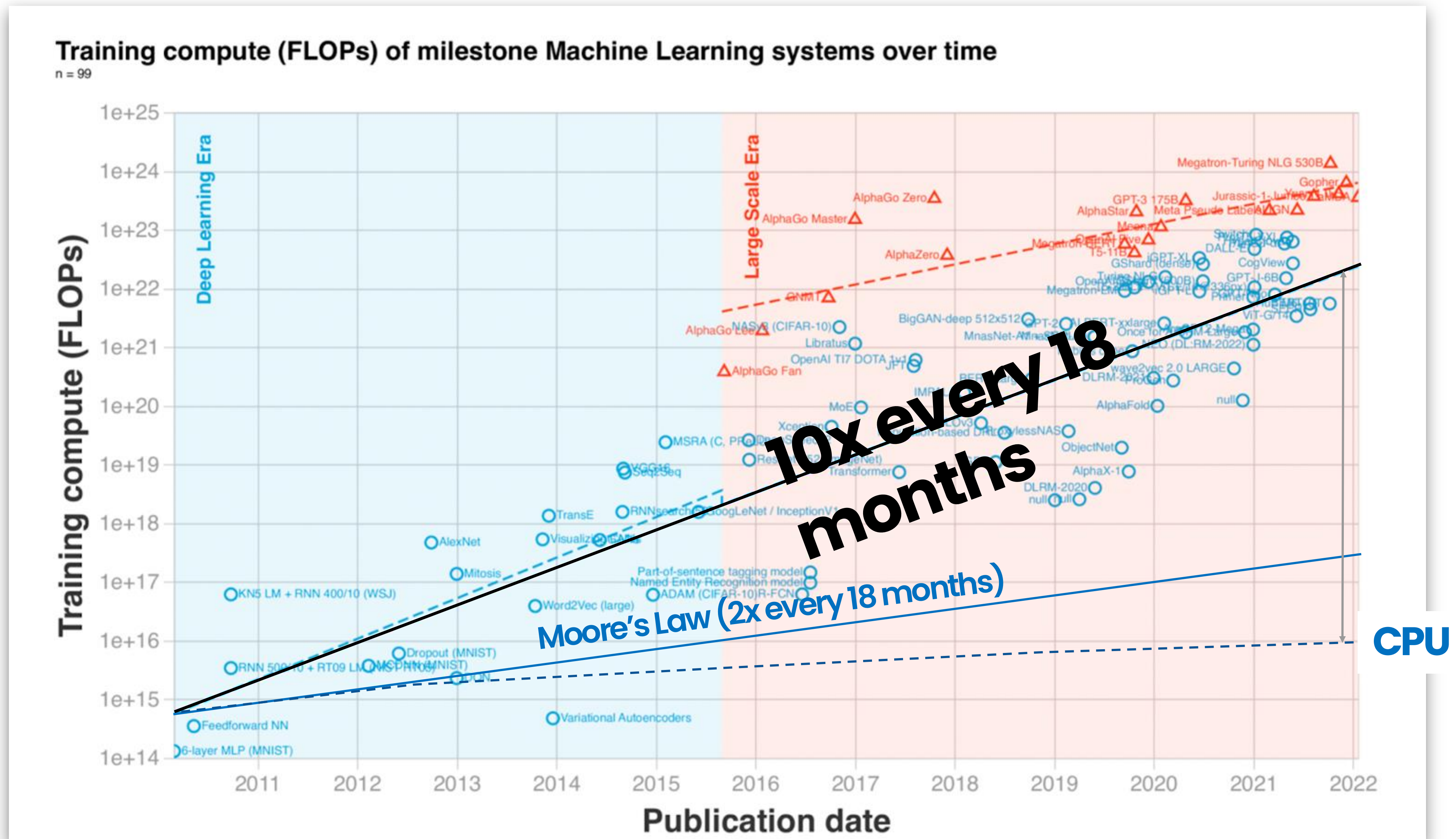
Reality Check: ML Trend



Bigger model, better accuracy



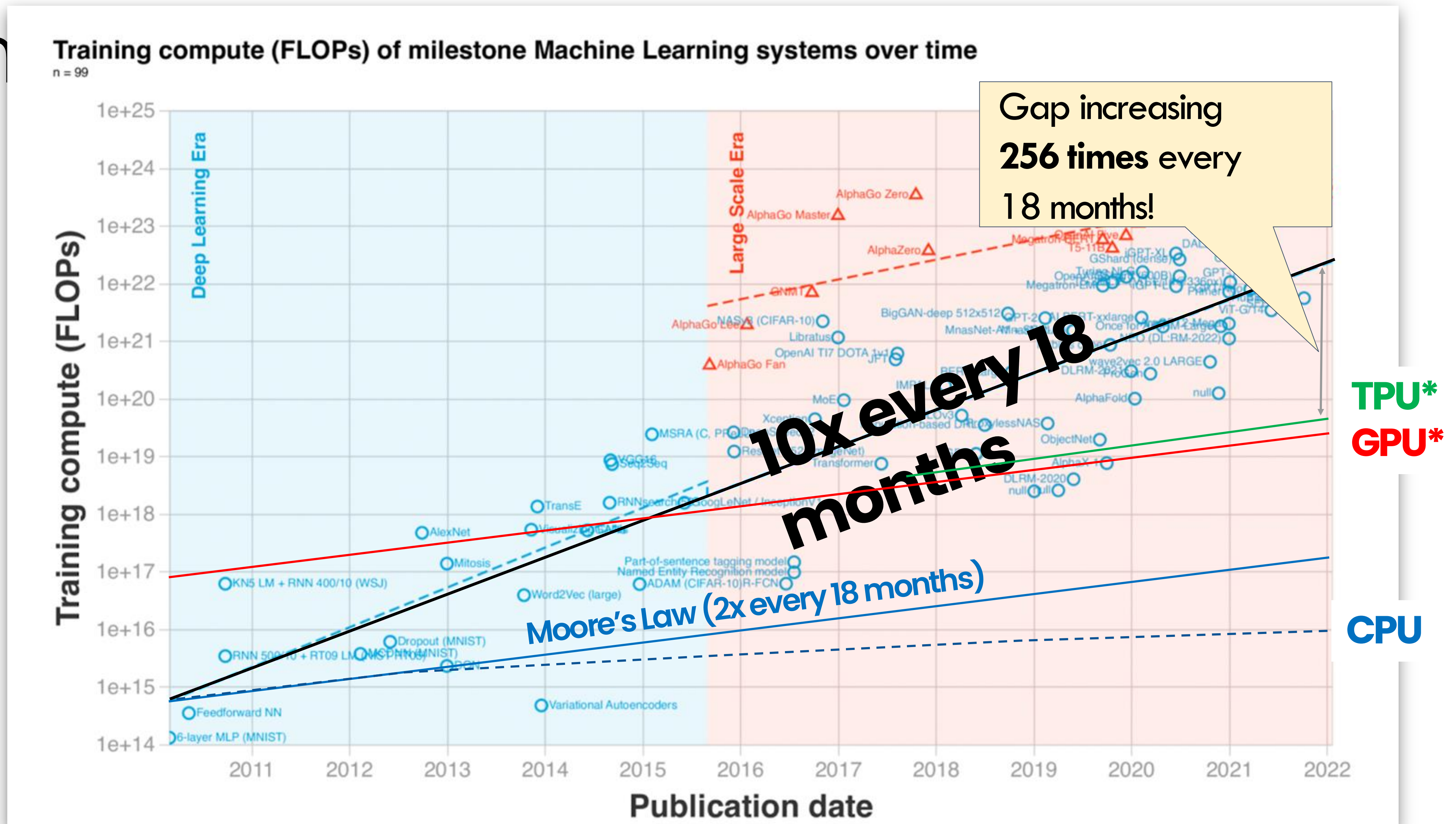
Growing gap between demand and supply



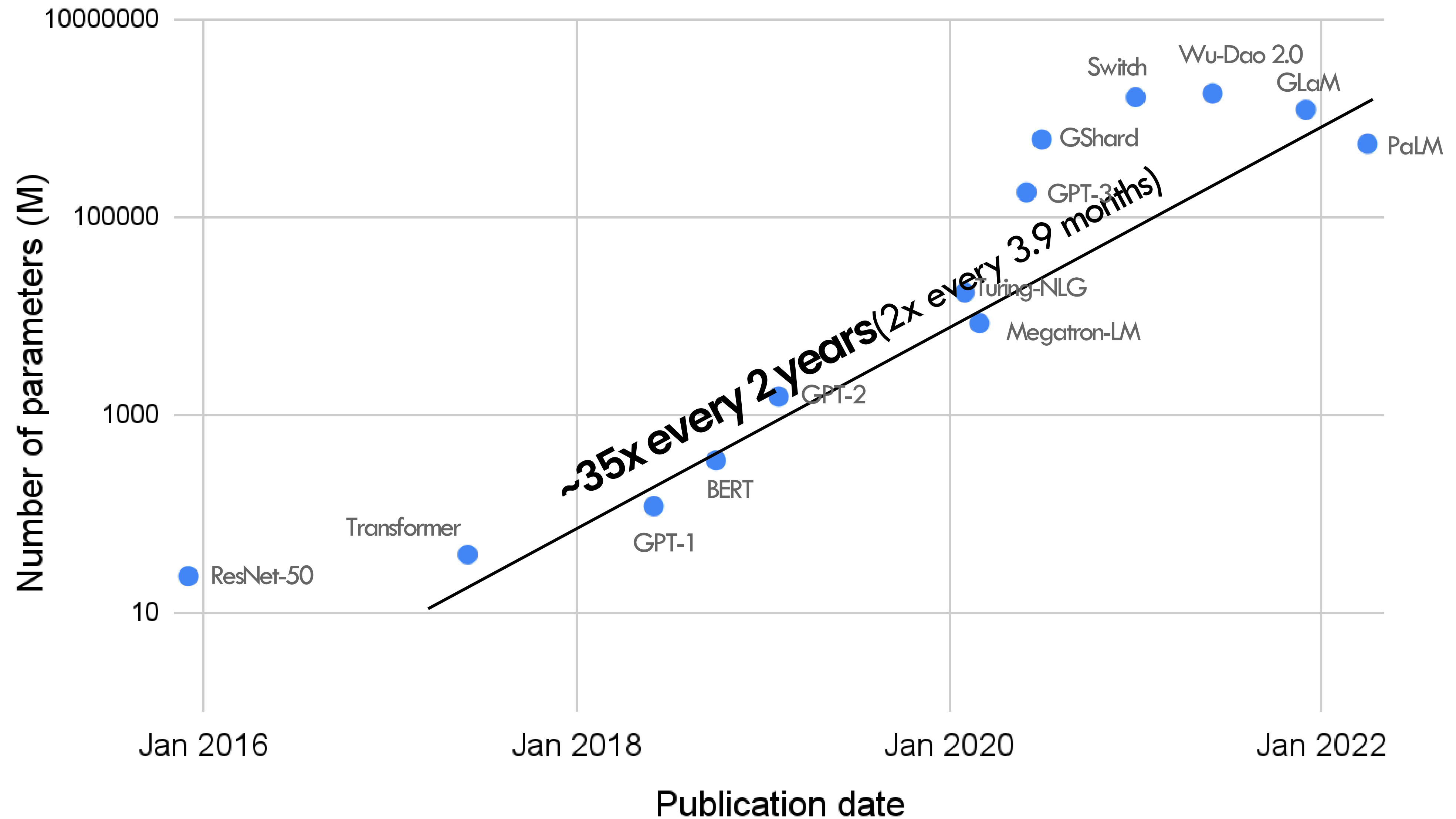
“Compute trends across three eras of machine learning”, J. Sevilla, <https://arxiv.labs.arxiv.org/html/2202.05924>

Specialized hardware not good

en



Not only compute, but memory



**No way out but to parallelize
these workloads !**

Parallelization

- Why Parallelization: Technology Trend
- **ML Parallelism Overview**
- Collective Communication Overview
- Inter and intra-op parallelism

Dataflow Graph

Autodiff

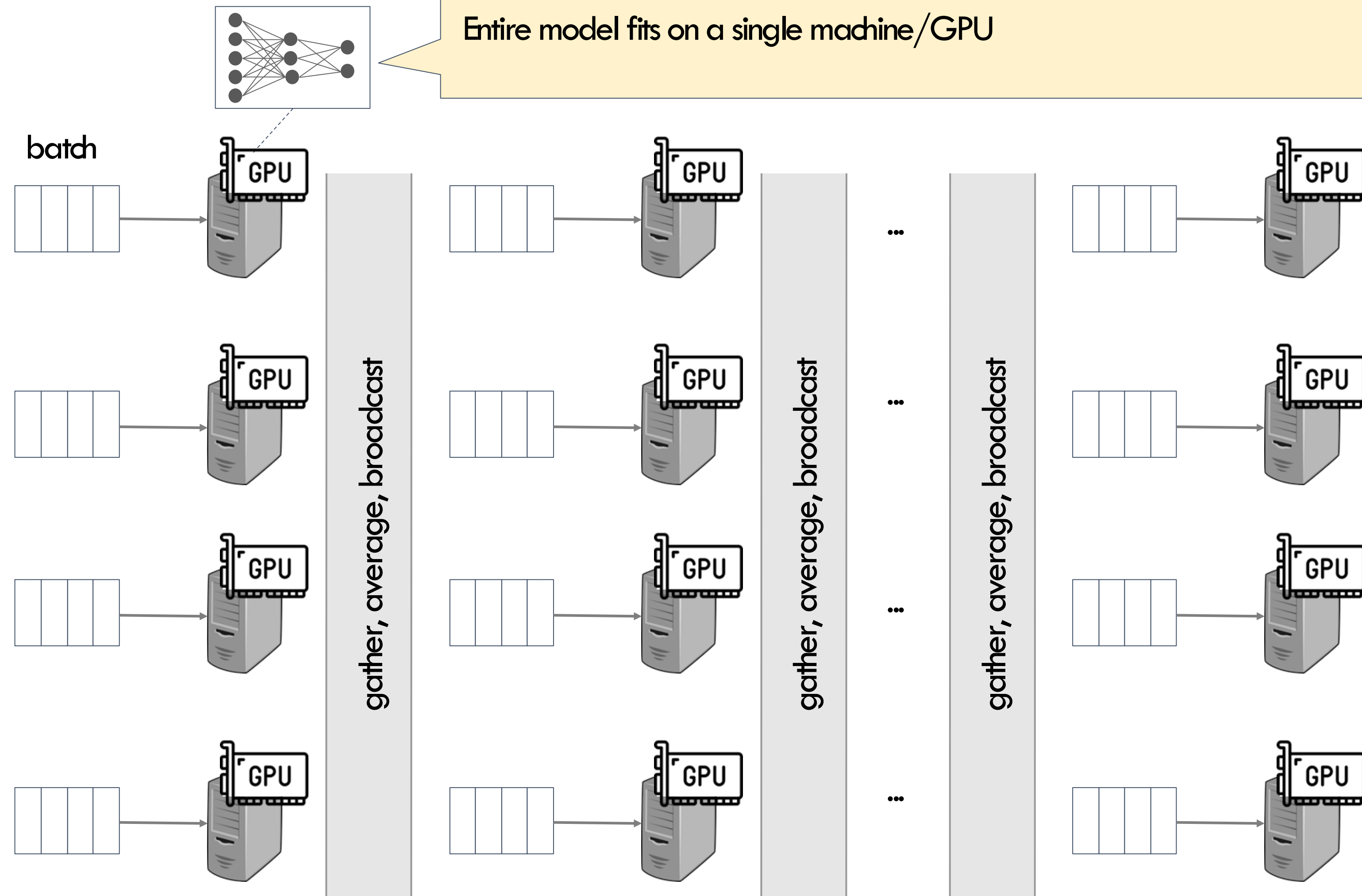
Graph Optimization

Parallelization

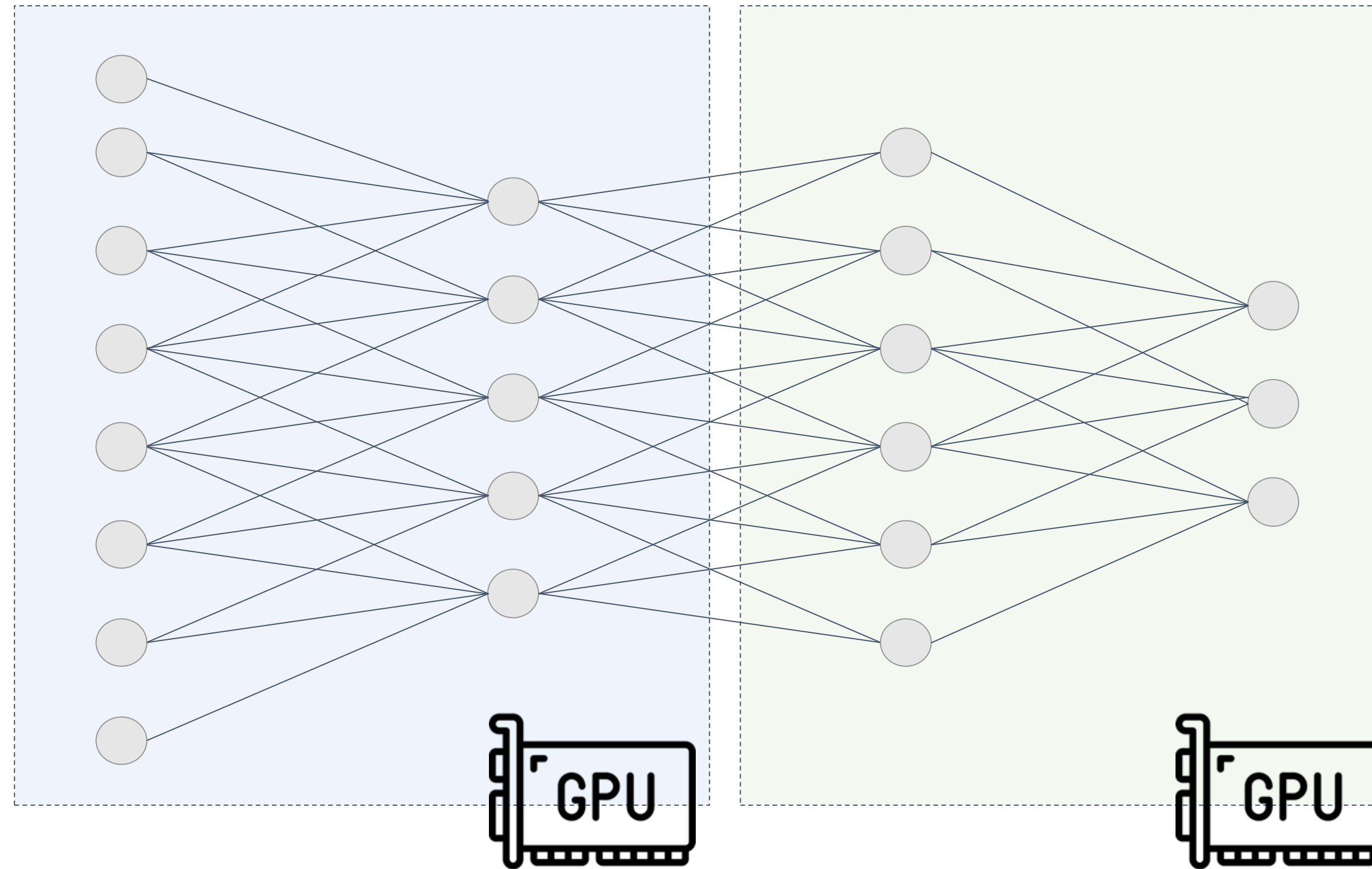
Runtime

Operator

Data Parallel Training

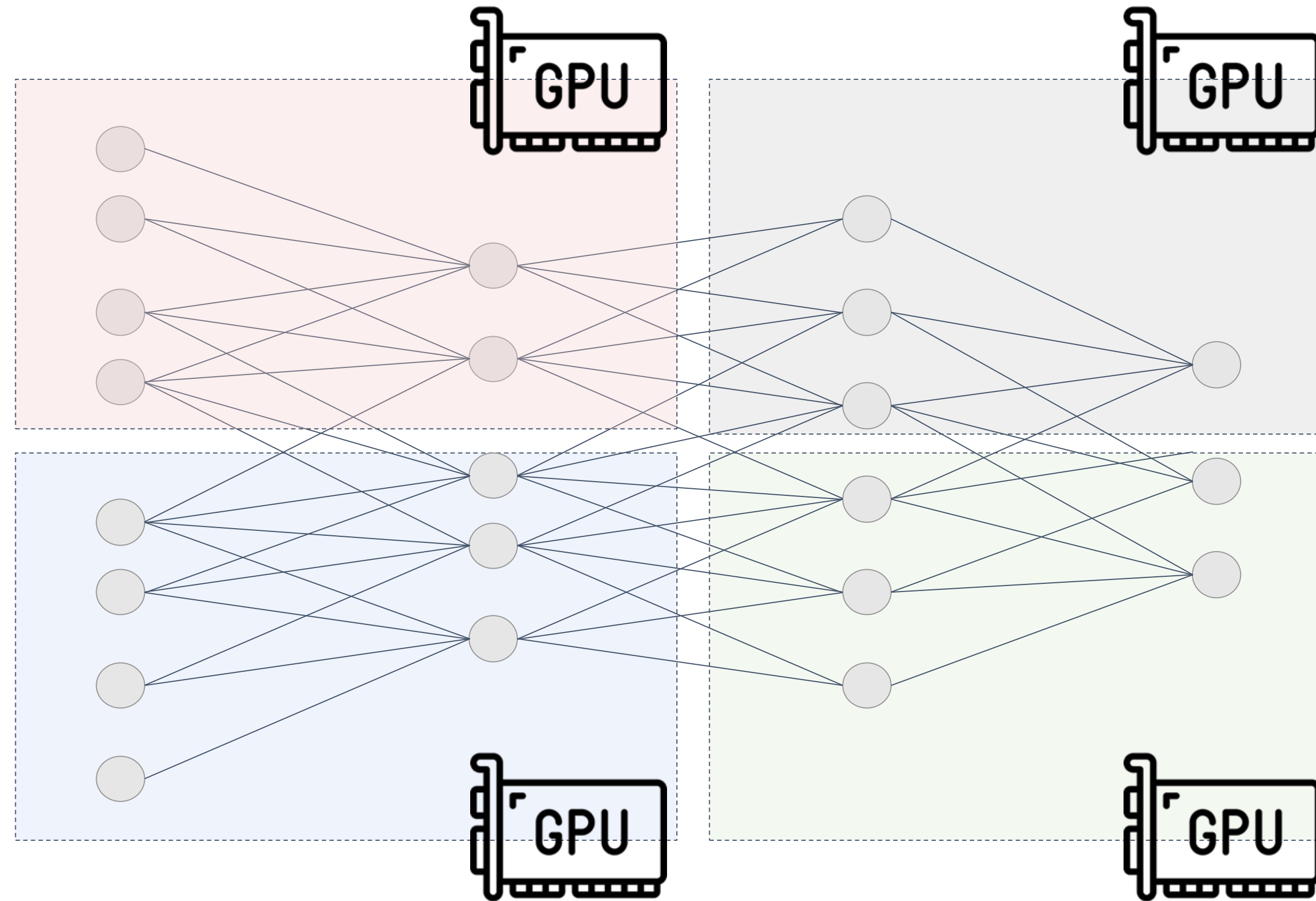


Model Parallelism



- Pipeline execution on both forward and backward paths
- GPUs can be on the same machine or **different** machines

Model Parallelism



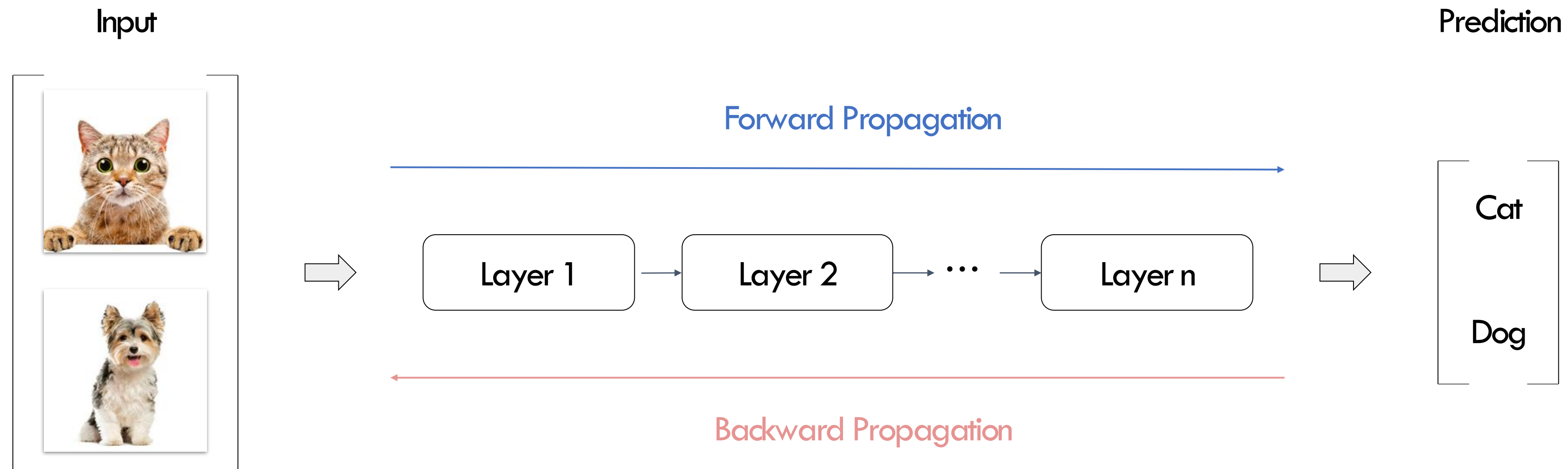
Classic View of ML Parallelisms

Classic view

Data parallelism

Model parallelism

From a Computational Perspective



$$\theta^{(t+1)} = f(\theta^{(t)}, \nabla_L(\theta^{(t)}, D^{(t)}))$$

parameter weight update (sgd, adam, etc.) model (CNN, GPT, etc.) data

Recap: Computational Graph

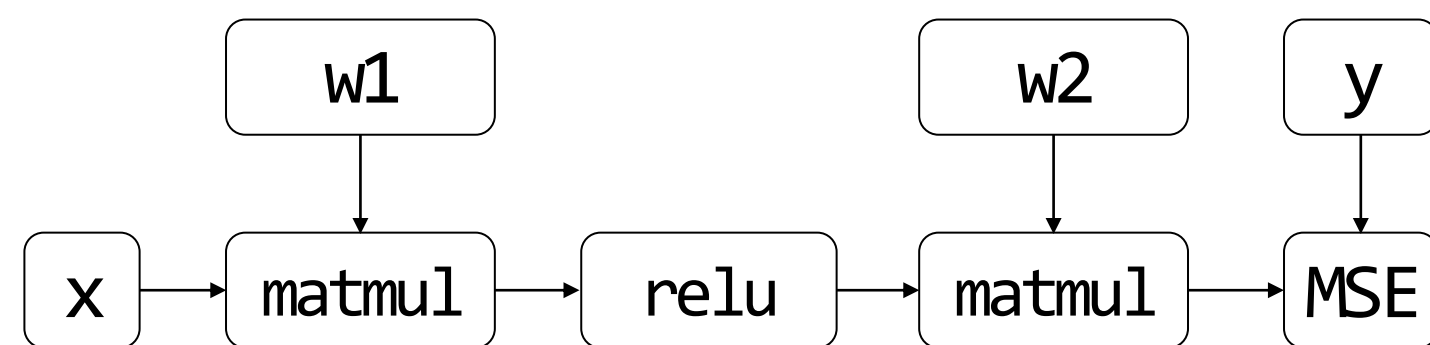
$$\theta^{(t+1)} = f(\theta^{(t)}, \nabla_L(\theta^{(t)}, D^{(t)}))$$

$$L = \text{MSE}(w_2 \cdot \text{ReLU}(w_1 x), y) \quad \theta = \{w_1, w_2\}, D = \{(x, y)\}$$

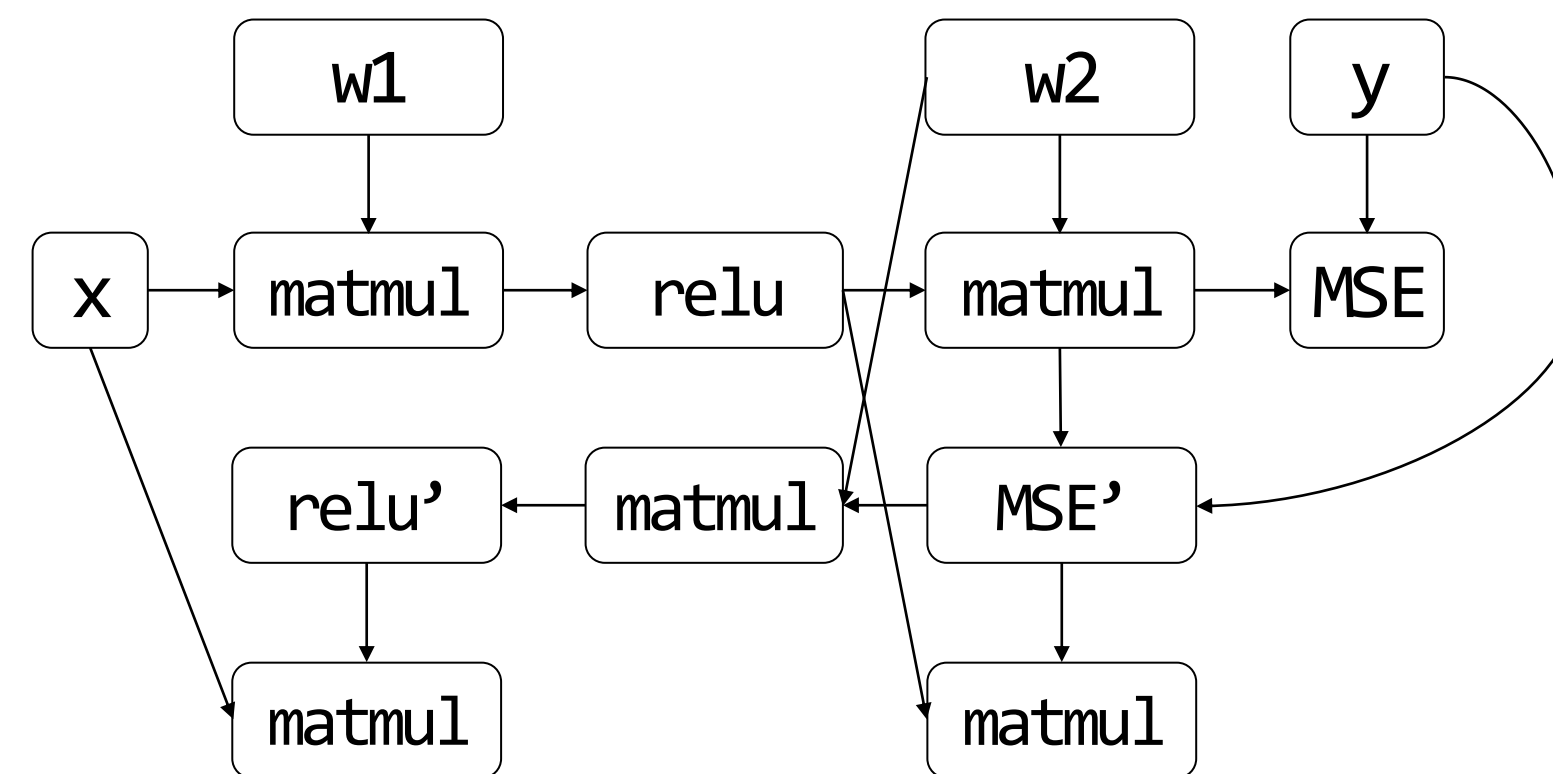
$$f(\theta, \nabla_L) = \theta - \nabla_L$$

 Operator / its output tensor $\longrightarrow$ Data flowing direction

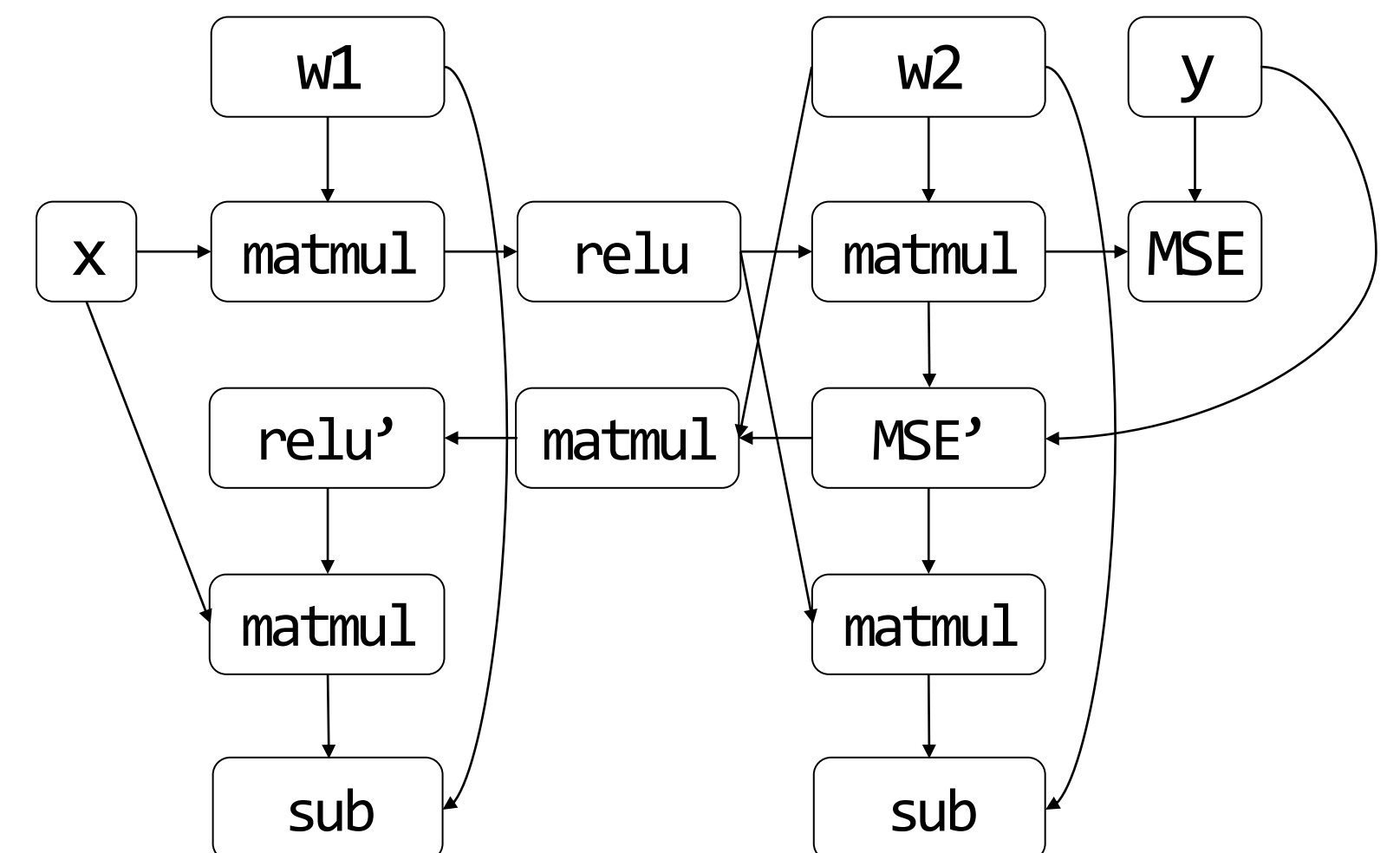
Forward



+Backward



+Weight update



Device Cluster

Nvidia DGX with V100

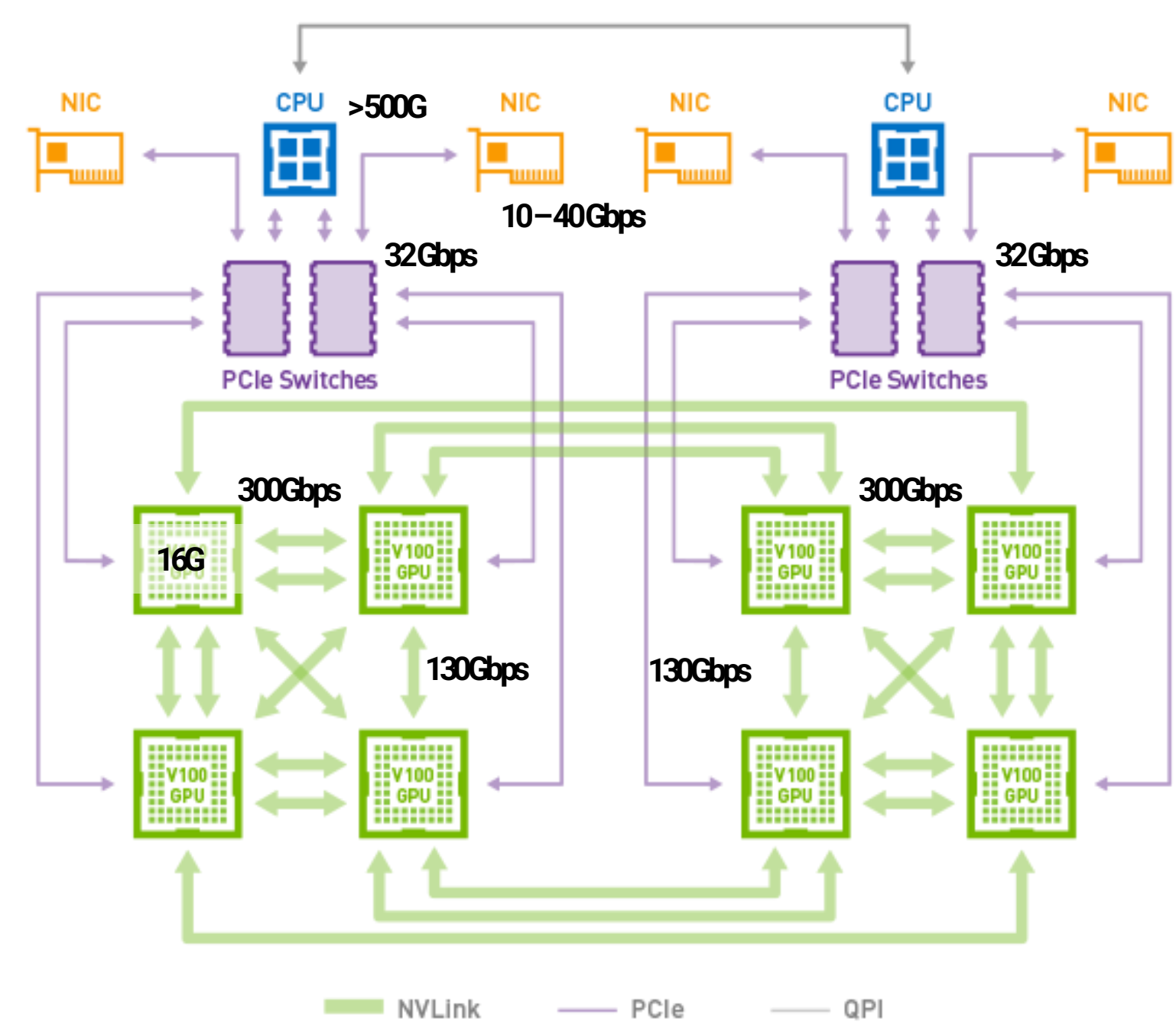
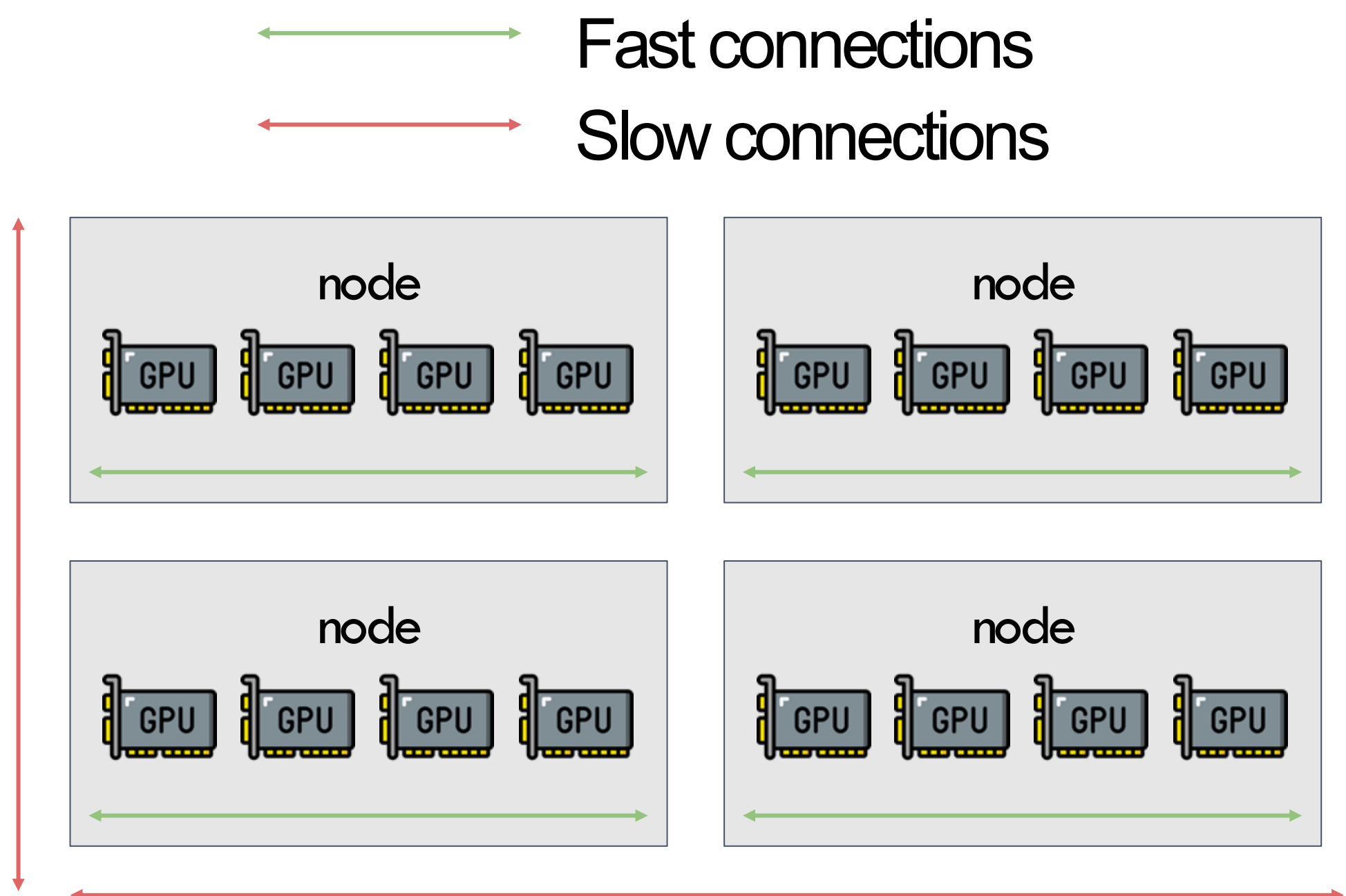


Figure from NVIDIA

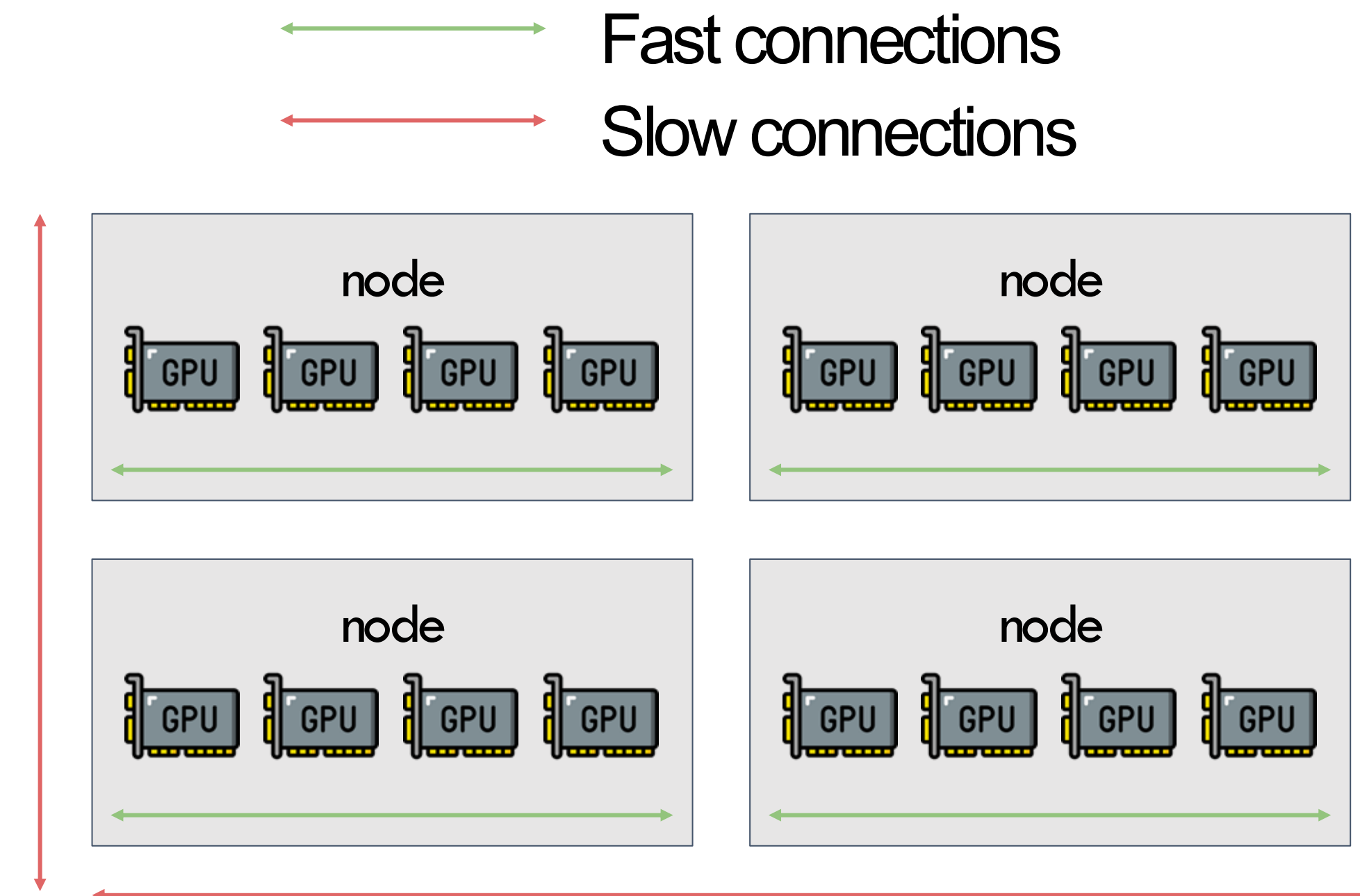
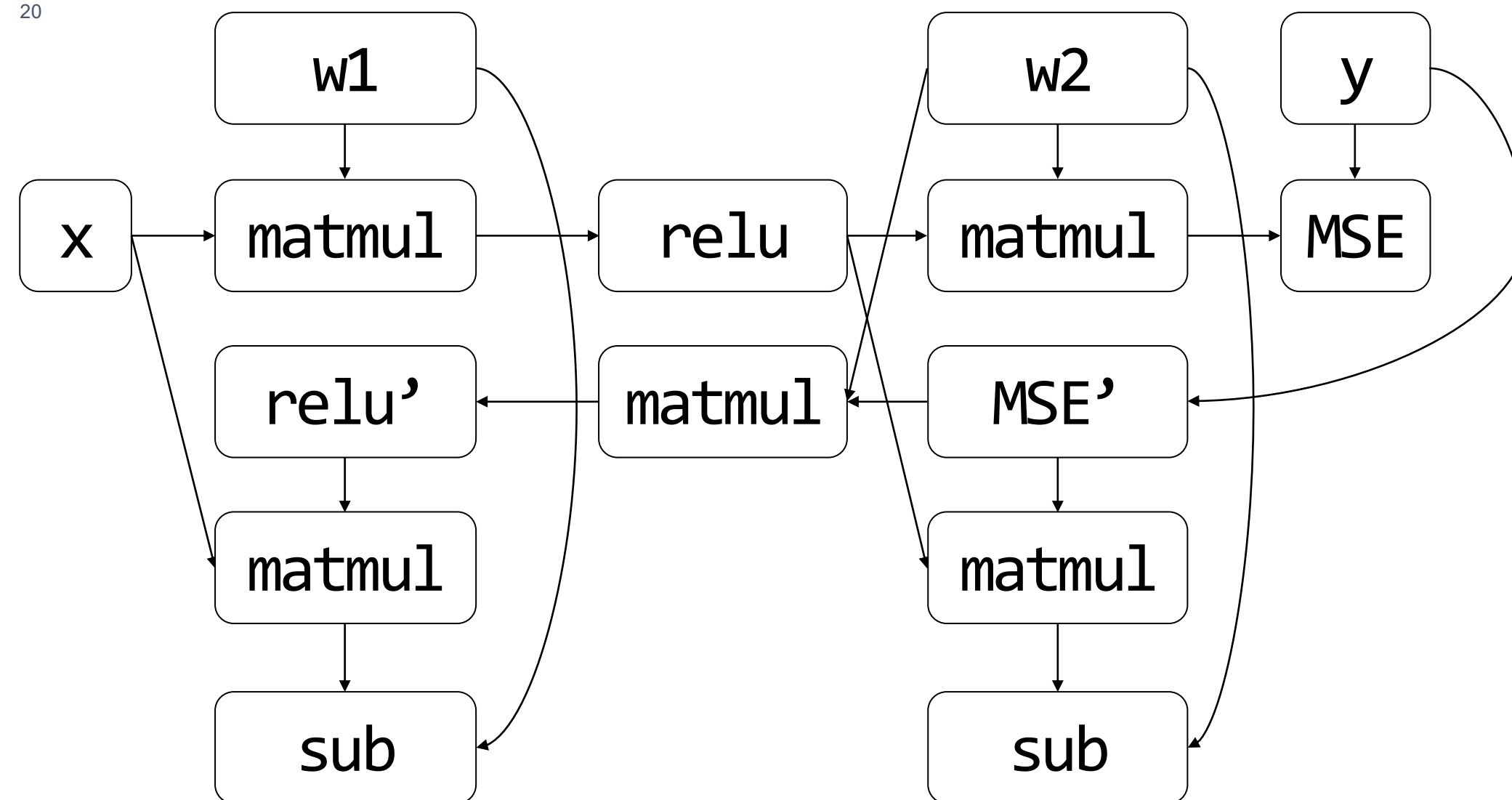
A typical GPU cluster topology



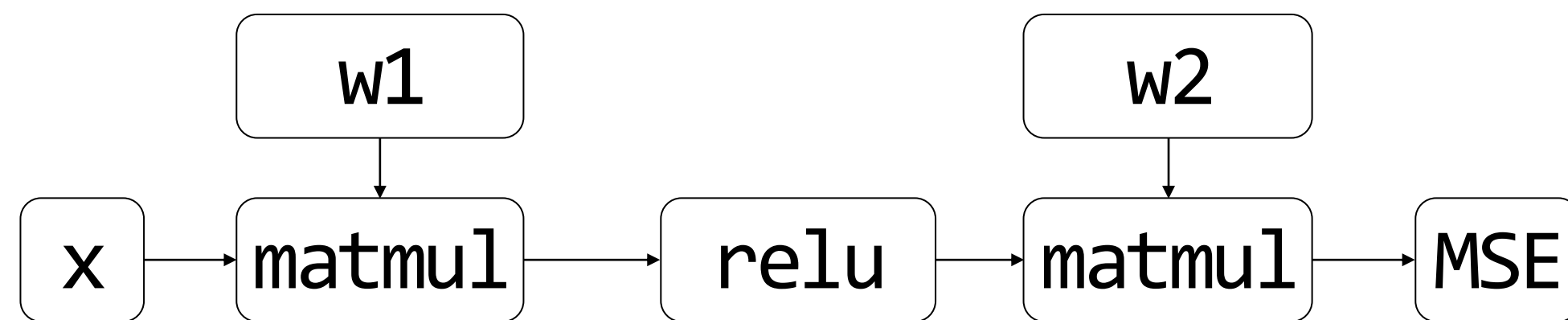
Parallelization = Partitioning Computation Graph on Device Cluster

How to partition the computational graph on the device cluster?

20



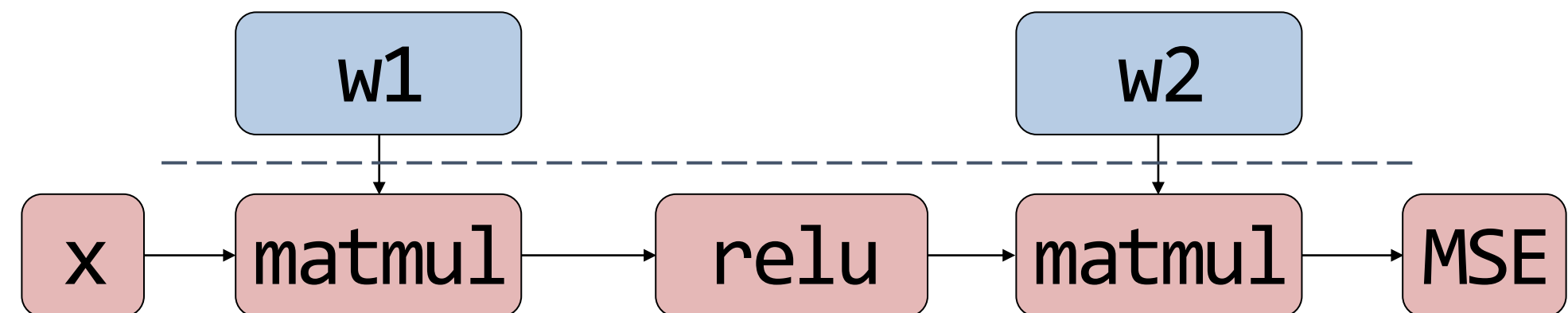
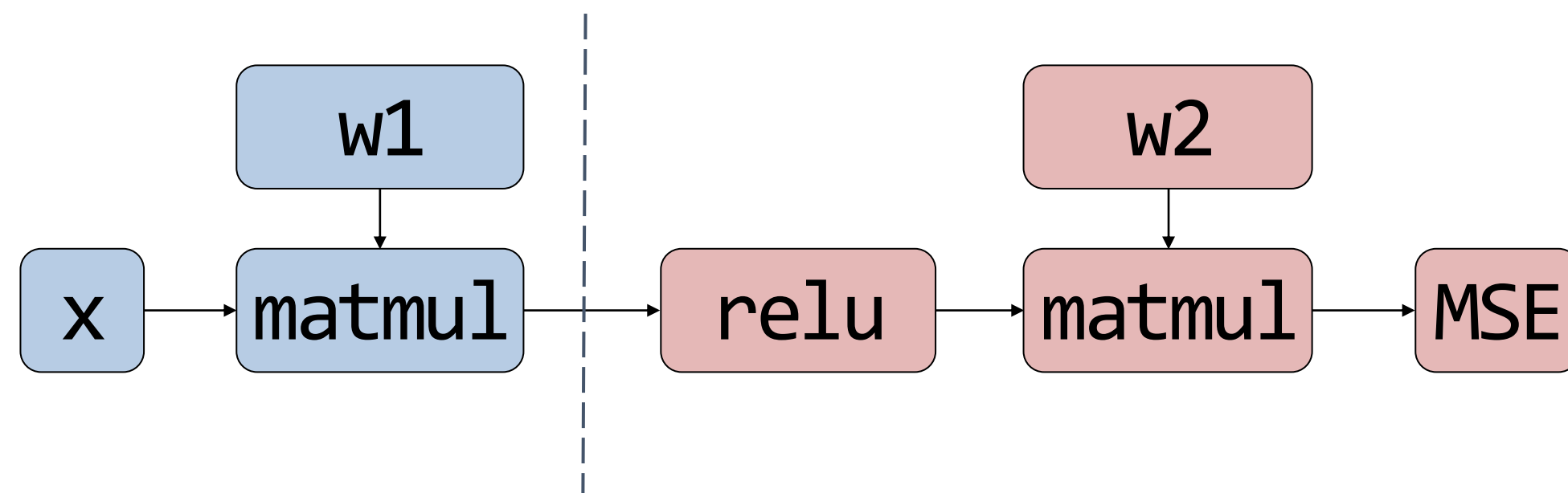
Partitioning Computation Graph



Device 1

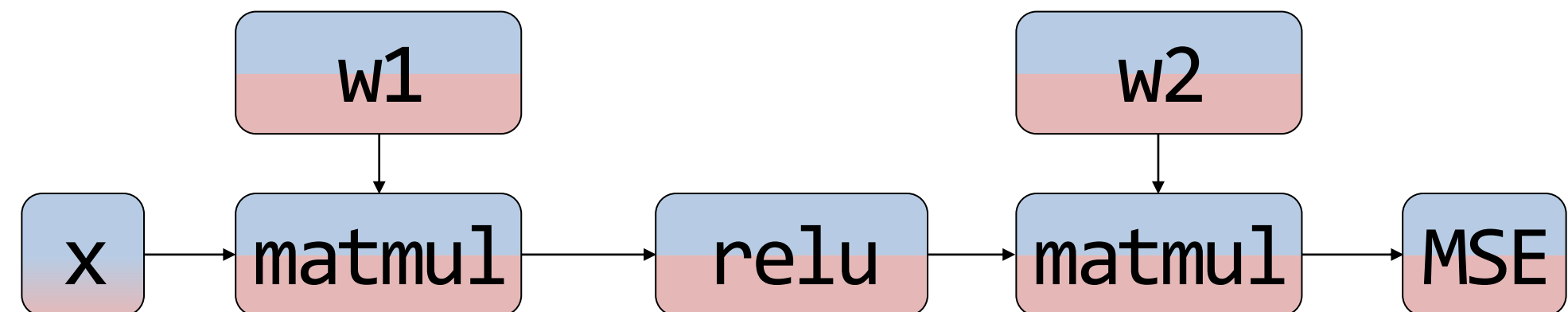
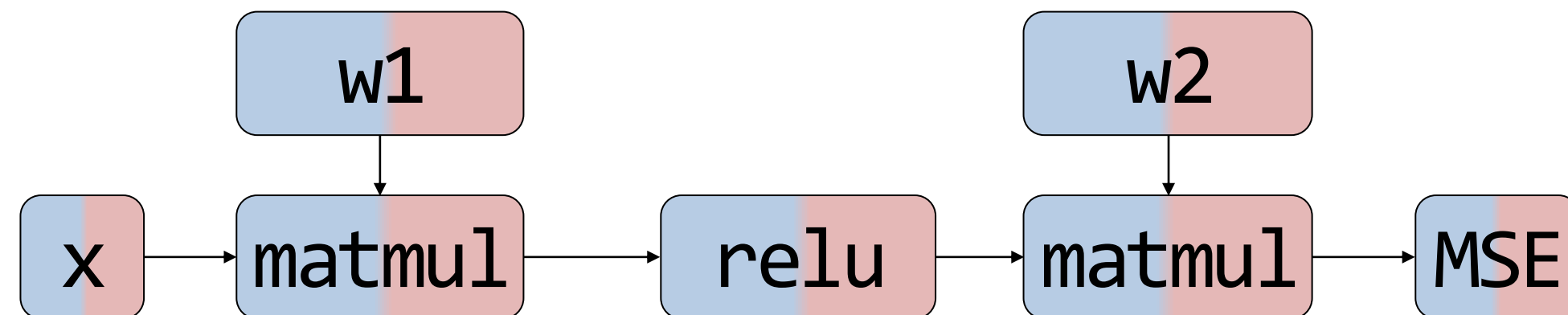
Device 2

Strategy 1



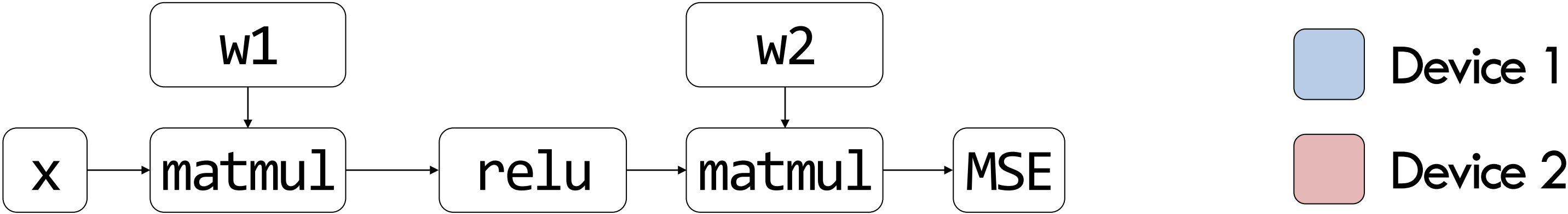
...

Strategy 2

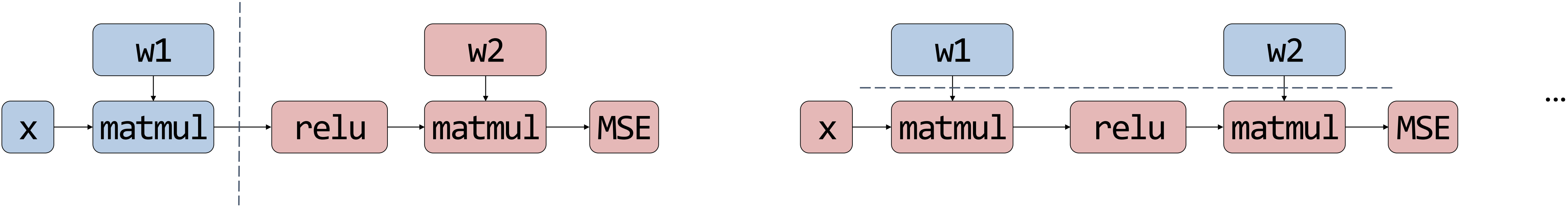


...

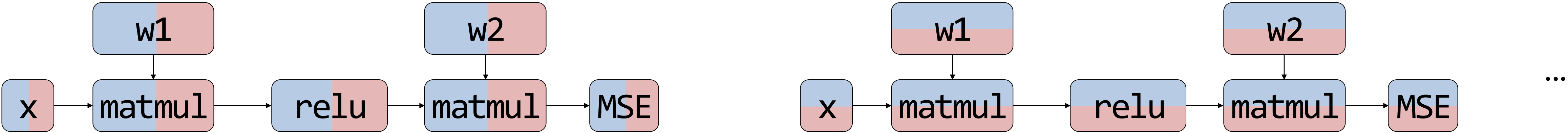
Partitioning Computation Graph



Strategy 1: Inter-operator Parallelism

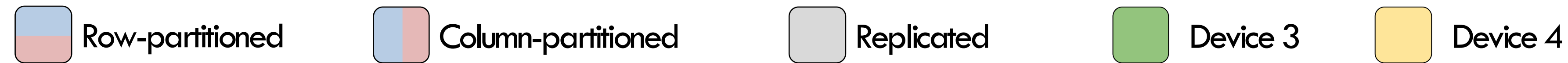


Strategy 2: Intra-operator Parallelism

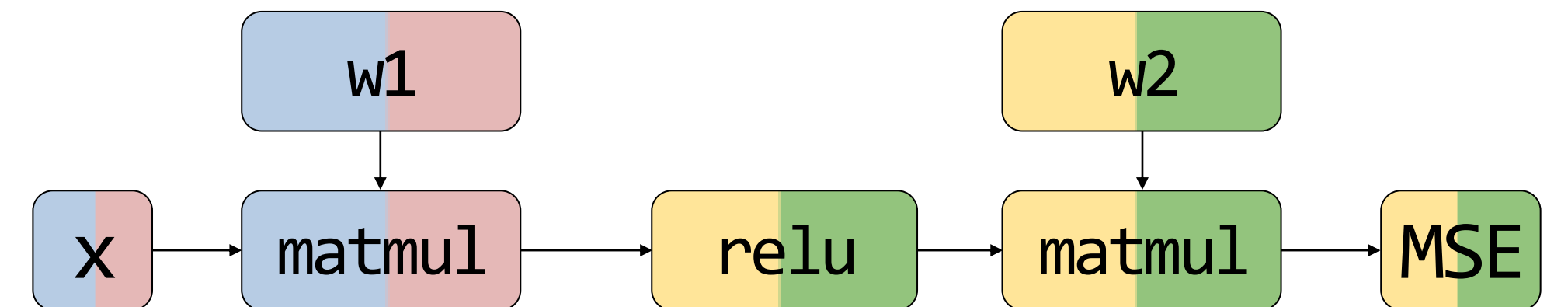
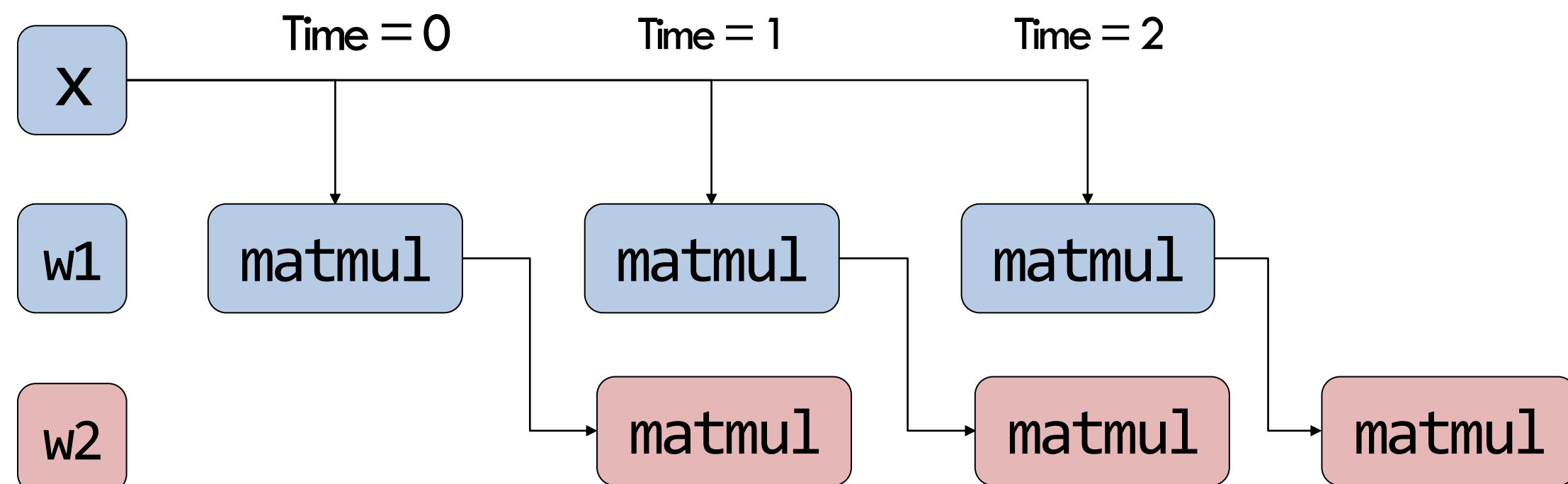
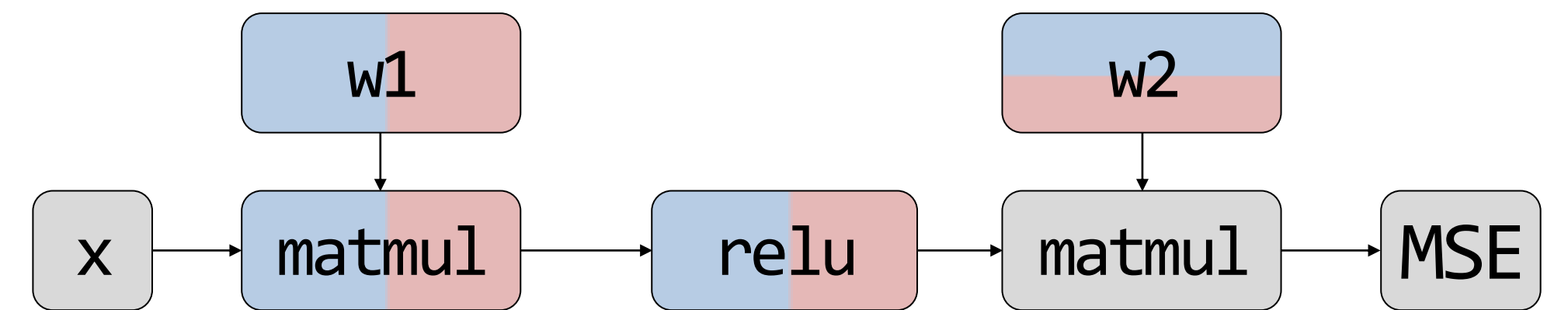
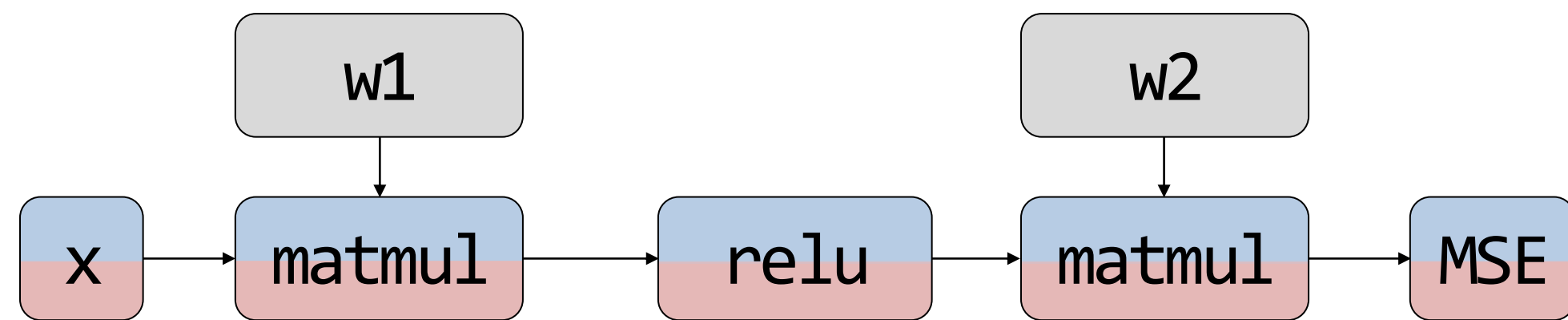


More Parallelisms...

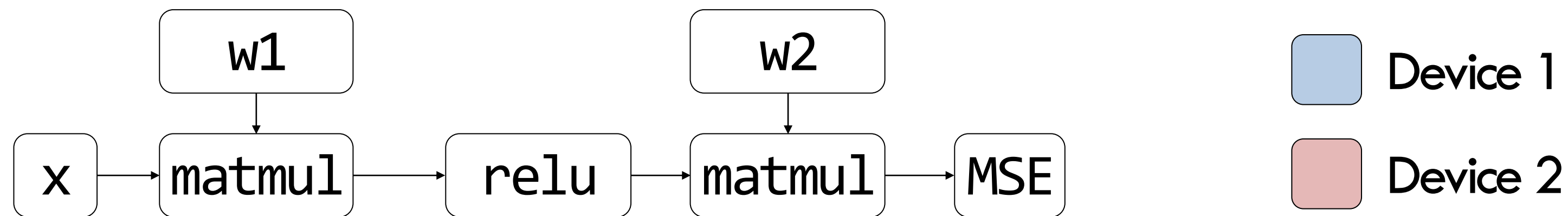
Multiple intra-op strategies for a single node



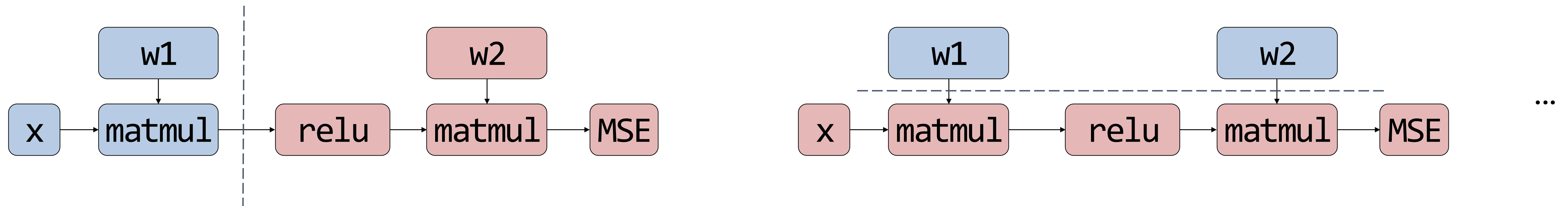
More strategies



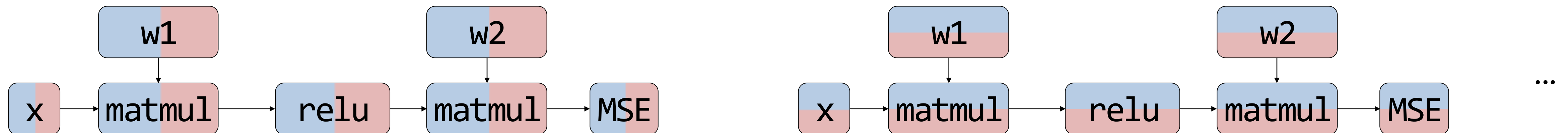
Summary: Inter-op and Intra-op Parallelisms



Inter-op parallelism: Assign different operators to different devices.



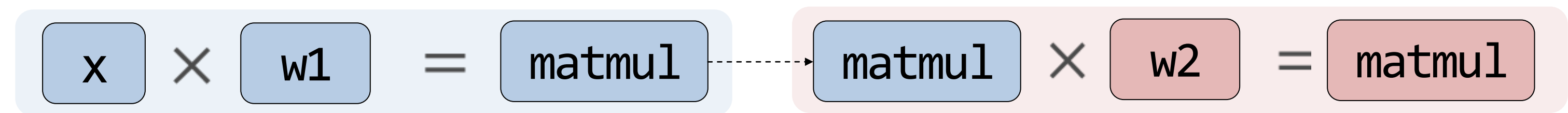
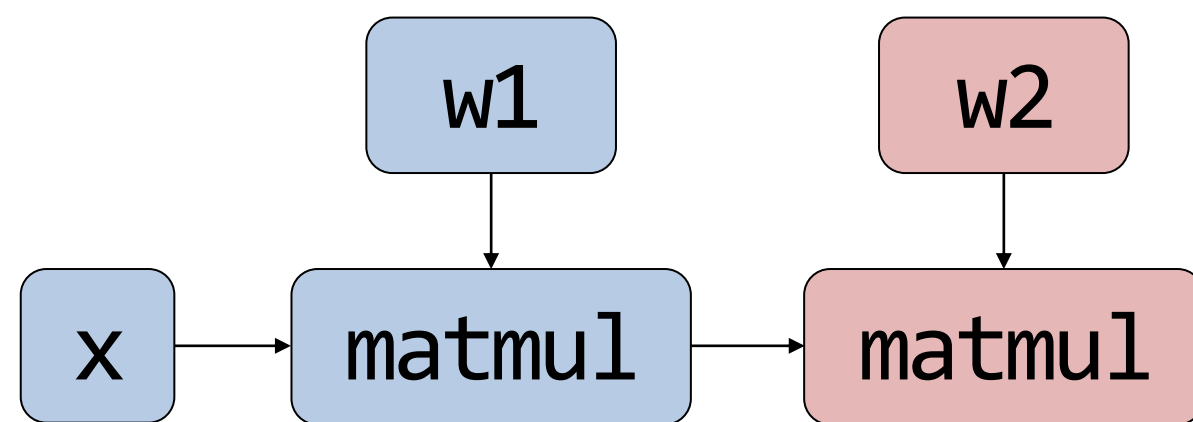
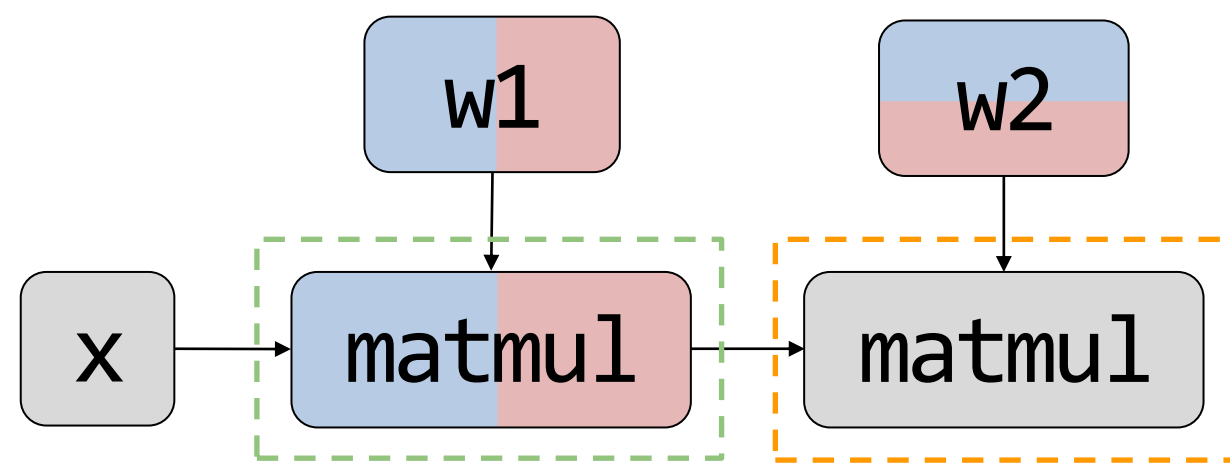
Intra-op parallelism: Assign different regions of a single operator to different devices.



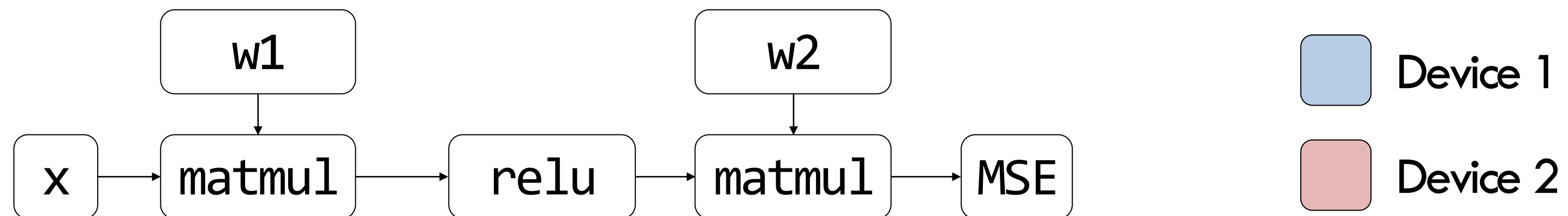
Inside Intra- and Inter-op Parallelism



$$Y = X \cdot W_1 \cdot W_2 = X \cdot \begin{bmatrix} W_1^{d1} & W_1^{d2} \end{bmatrix} \cdot \begin{bmatrix} W_2^{d1} \\ W_2^{d2} \end{bmatrix}$$

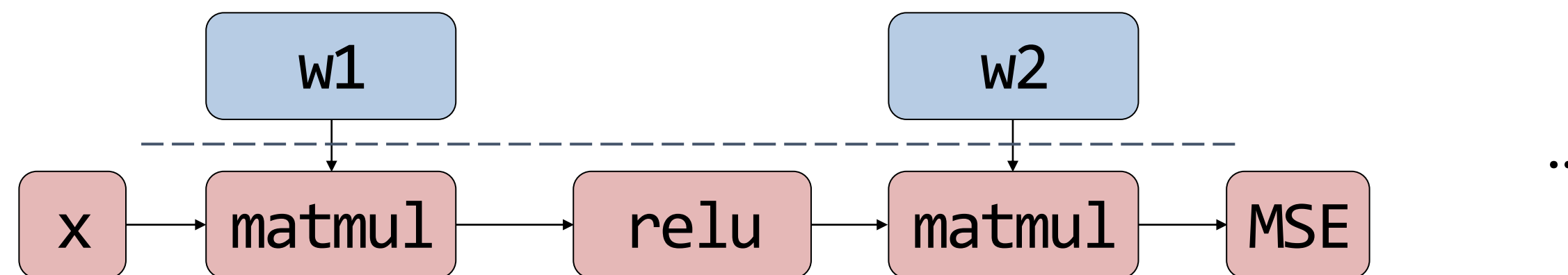
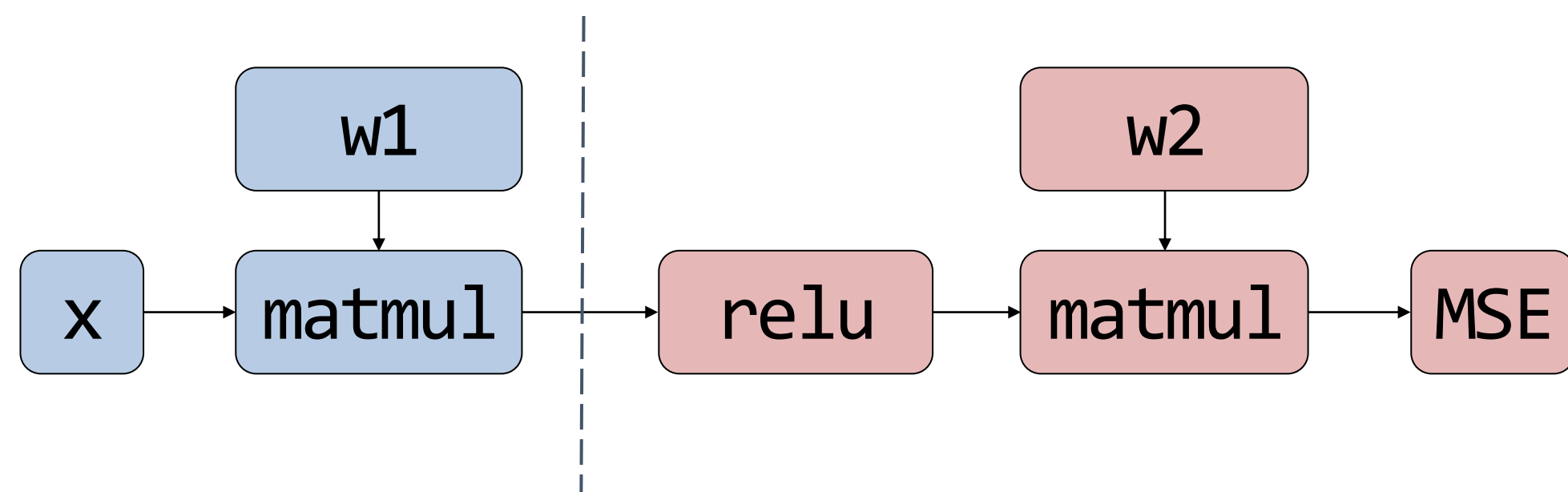


Inter-op and Intra-op Parallelism: Characteristics



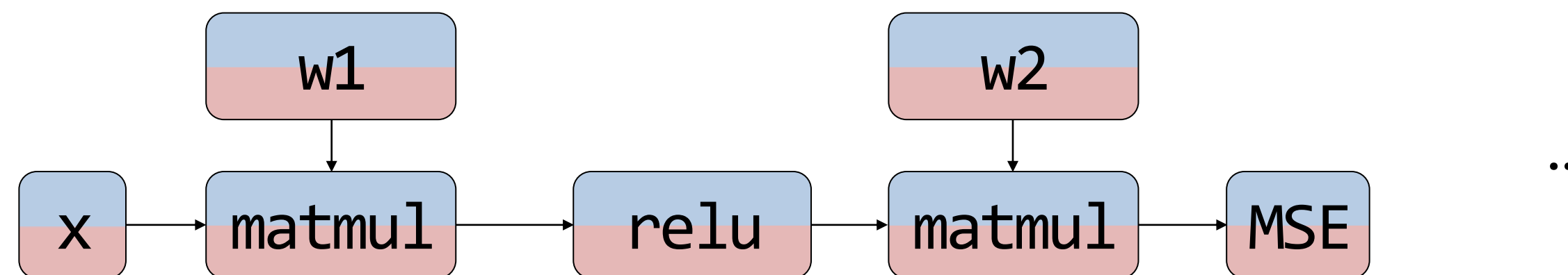
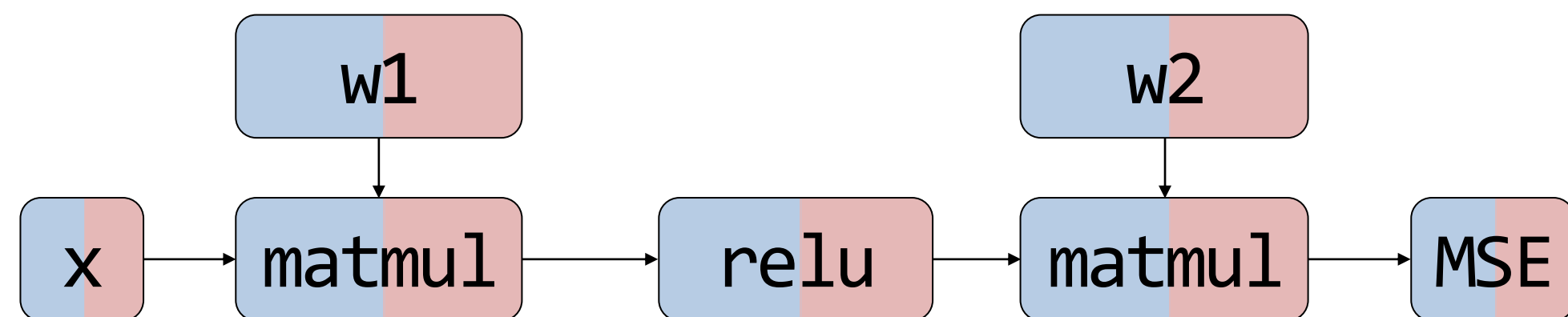
Inter-op parallelism:

Requires point-to-point communication but results in device idle

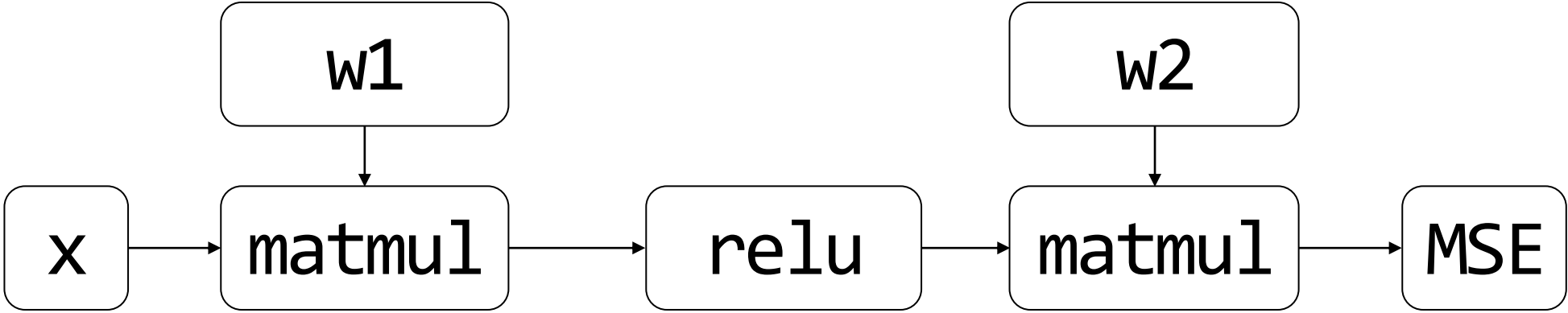


Intra-op parallelism:

Devices are busy but requires collective communication



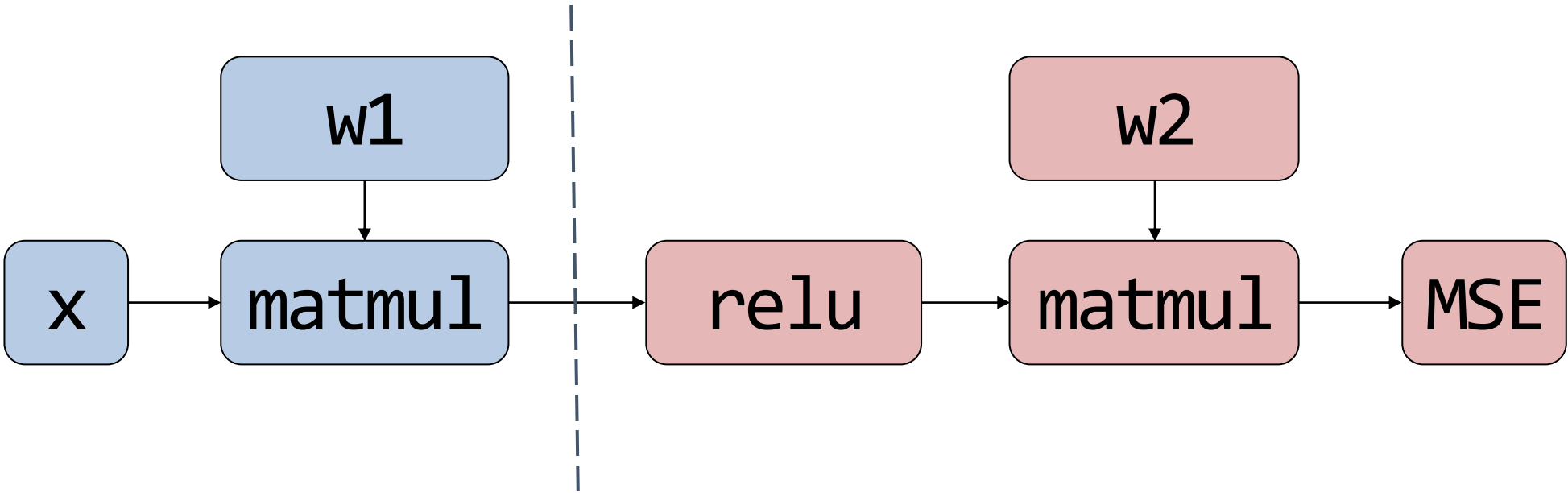
Inter-op and Intra-op Parallelism: Characteristics



Device 1

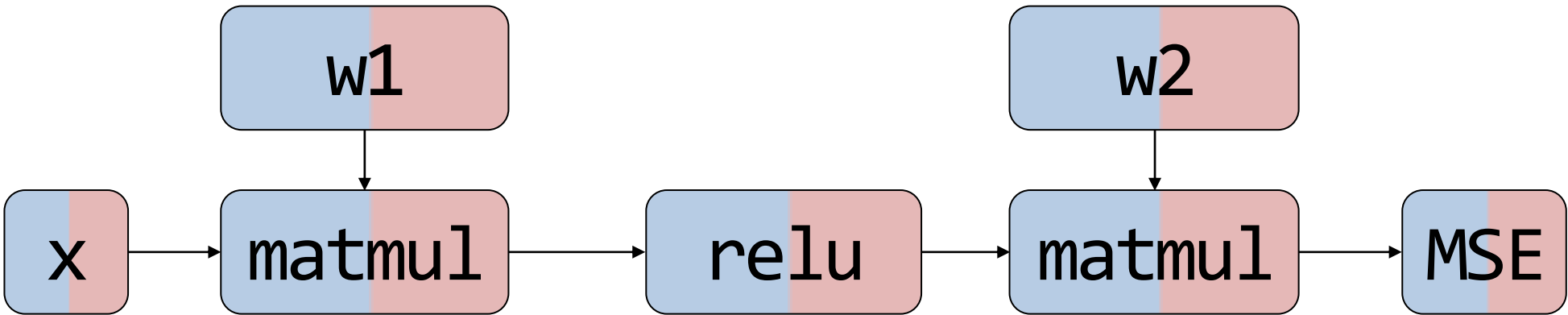
Device 2

Inter-op parallelism



Trade-off

Intra-op parallelism



	Inter-operator Parallelism	Intra-operator Parallelism
Communication	Less	More
Device Idle Time	More	Less

Computational View of ML Parallelisms

Classic view

Data parallelism

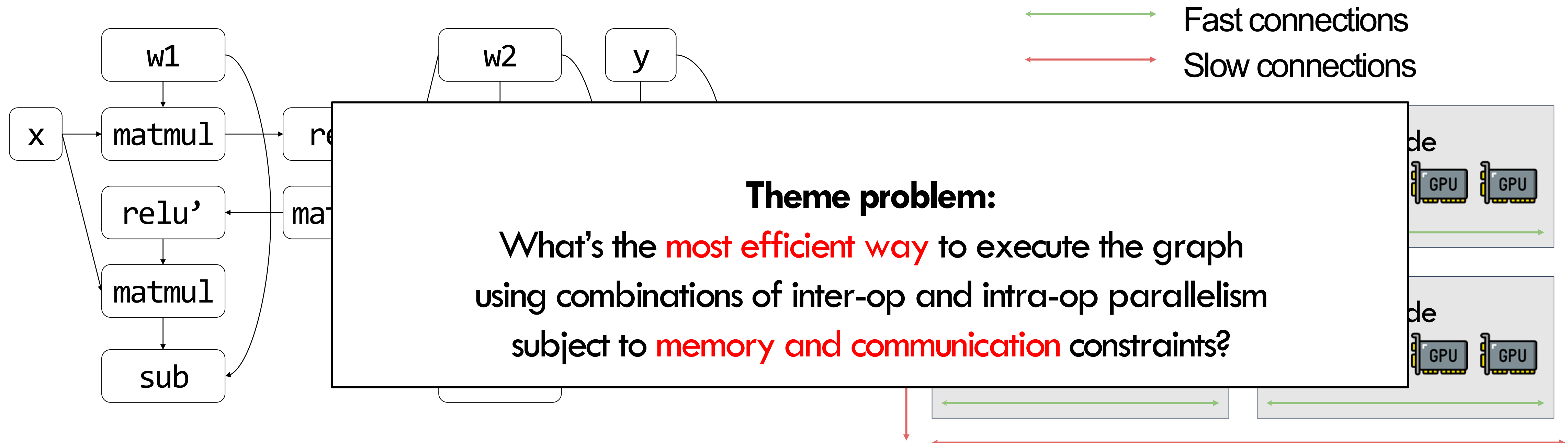
Model parallelism

New view (this course)

Inter-op parallelism

Intra-op parallelism

ML Parallelization under New View



How to Measure Efficiency of Parallelism?

$$\text{max AI} = \#ops / \#bytes$$

A More Holistic (Macro) Measure: MFU)

H100 SXM	
FP64	34 teraFLOPS
FP64 Tensor Core	67 teraFLOPS
FP32	67 teraFLOPS
TF32 Tensor Core*	989 teraFLOPS
BFLOAT16 Tensor Core*	1,979 teraFLOPS
FP16 Tensor Core*	1,979 teraFLOPS
FP8 Tensor Core*	3,958 teraFLOPS
INT8 Tensor Core*	3,958 TOPS

$$\text{MFU} = \frac{\text{\#FLOPs}}{t} / \text{peak FLOPS}$$

ML program's total flops

model flops utilization

time to finish the program

Potential Factors Compromising MFU

$$\text{MFU} = \# \text{FLOPs} / t / \text{peak FLOPs}$$

- Op types in the computational graph (ml model type)
 - What are MFU-friendly op and MFU-unfriendly op?
- Optimization (which we have covered)
- Precision, core, and GPU type
- Communication over network
 - How to reduce/hide communication?

Potential Factors Lowing MFU

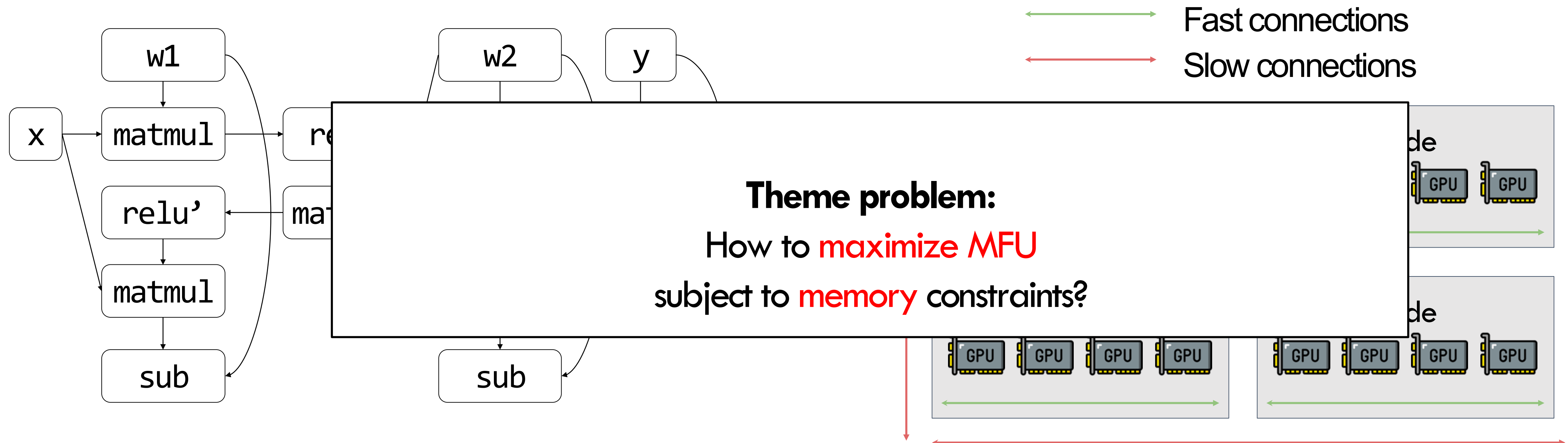
$$\text{MFU} = \# \text{FLOPs} / t / \text{peak FLOPs}$$

-  Op types, op shape (ml model type)
 - What are MFU-friendly op and MFU-unfriendly op?
-  Optimization (which we have covered)
-  Precision, core, and GPU type
-  Communication over network
 - How to reduce/hide communication?

MFU is becoming a metric highly indexed in LLM Industry

- V100: 40 – 50% MFU
 - V100: 112 TFLOPS
- A100: 40%
 - After flash attention: 60%
 - A100: 312 TFLOPS
- H100: 30 – 50% depending on model size
 - H100: 990 TFLOPs
- B100: where will it be?

Simplify the Problem



Parallelization

- Why Parallelization: Technology Trend
- ML Parallelism Overview
- **Collective Communication Overview**
- Inter and intra-op parallelism

Dataflow Graph

Autodiff

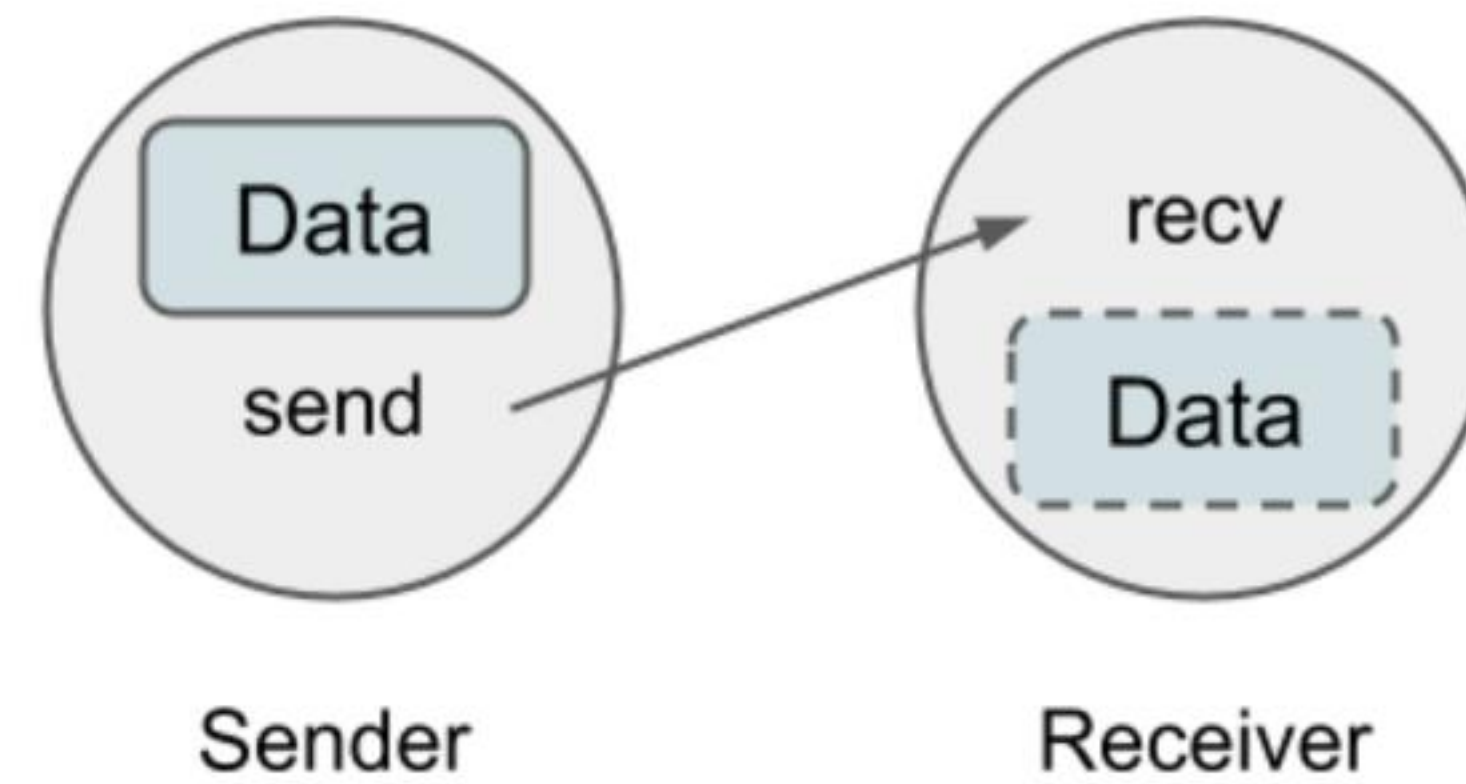
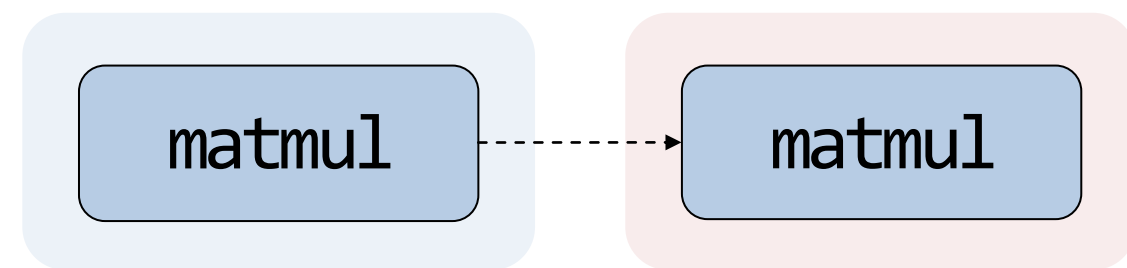
Graph Optimization

Parallelization

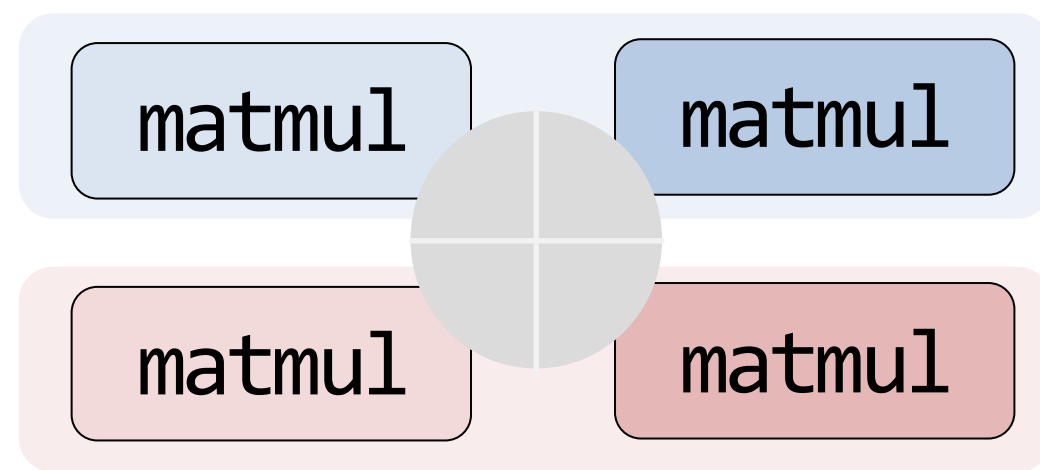
Runtime

Operator

Terminologies: Point-to-point Communication



Terminologies: Collective Communication



```
ddp_model = DDP(Model(), device_ids=[rank])
for batch in data_loader:
    loss = train_step(ddp_model, batch)
```

Implicit allreduce here

all-reduce

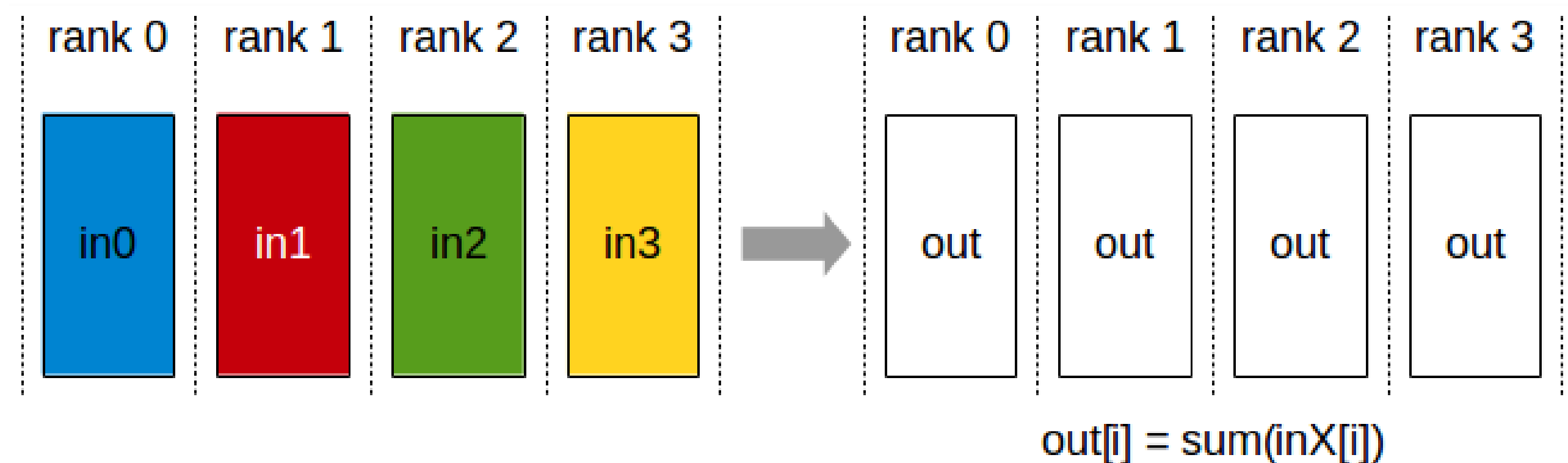


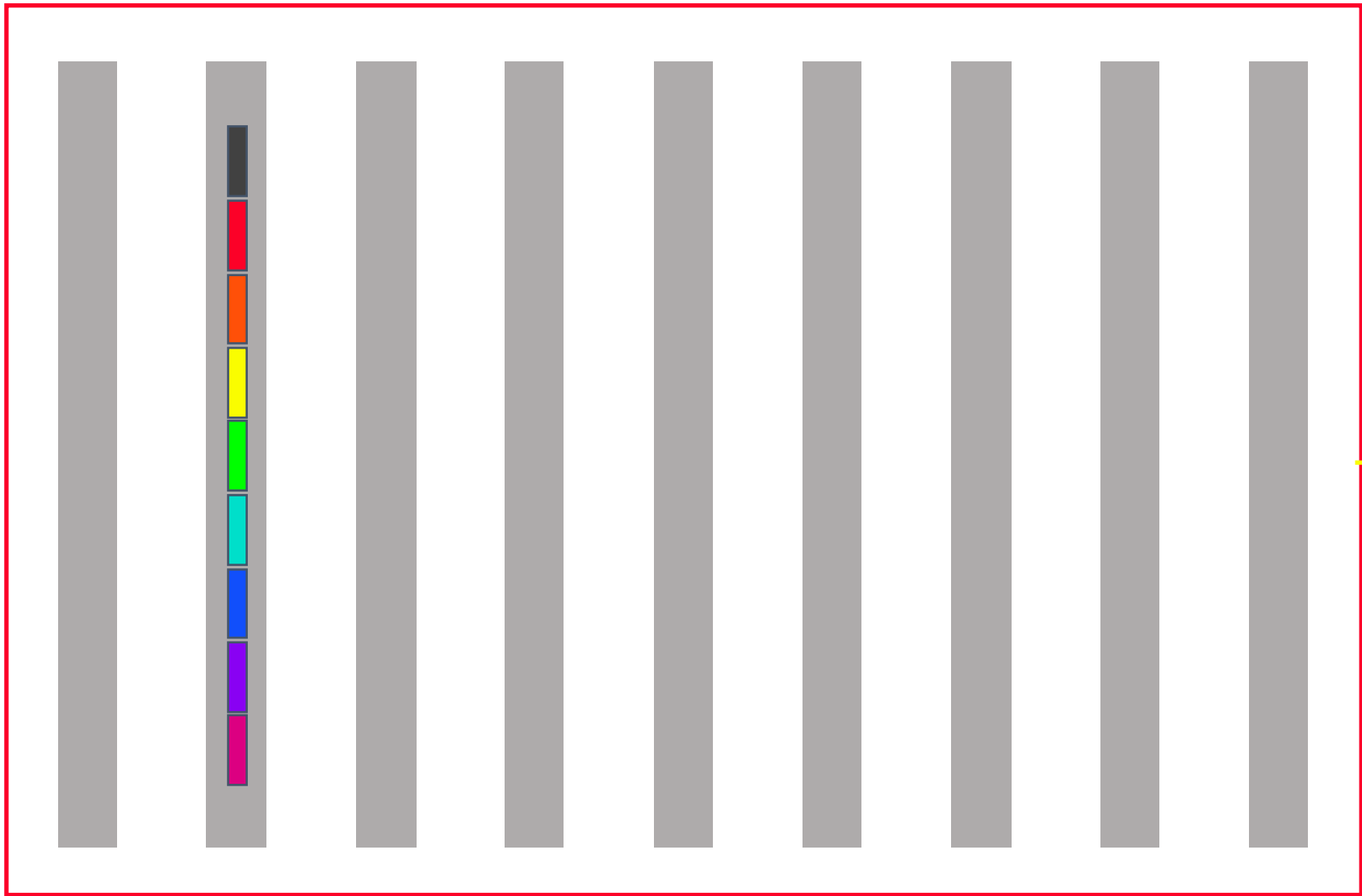
Figure from NCCL documentation

Collective Communications

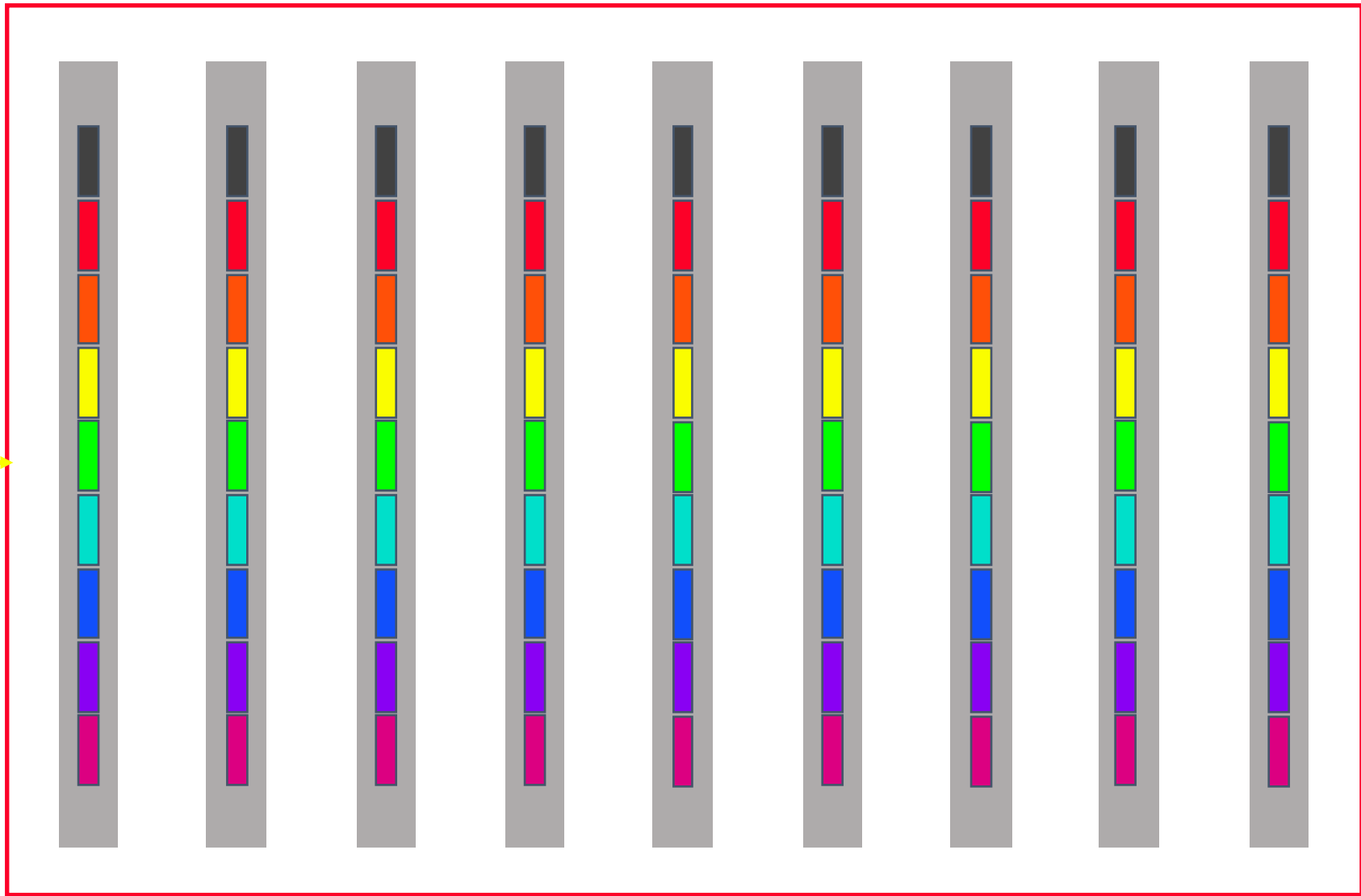
- Broadcast
- Reduce(-to-one)
- Scatter
- Gather
- Allgather
- Reduce-scatter
- Allreduce

Broadcast

Before

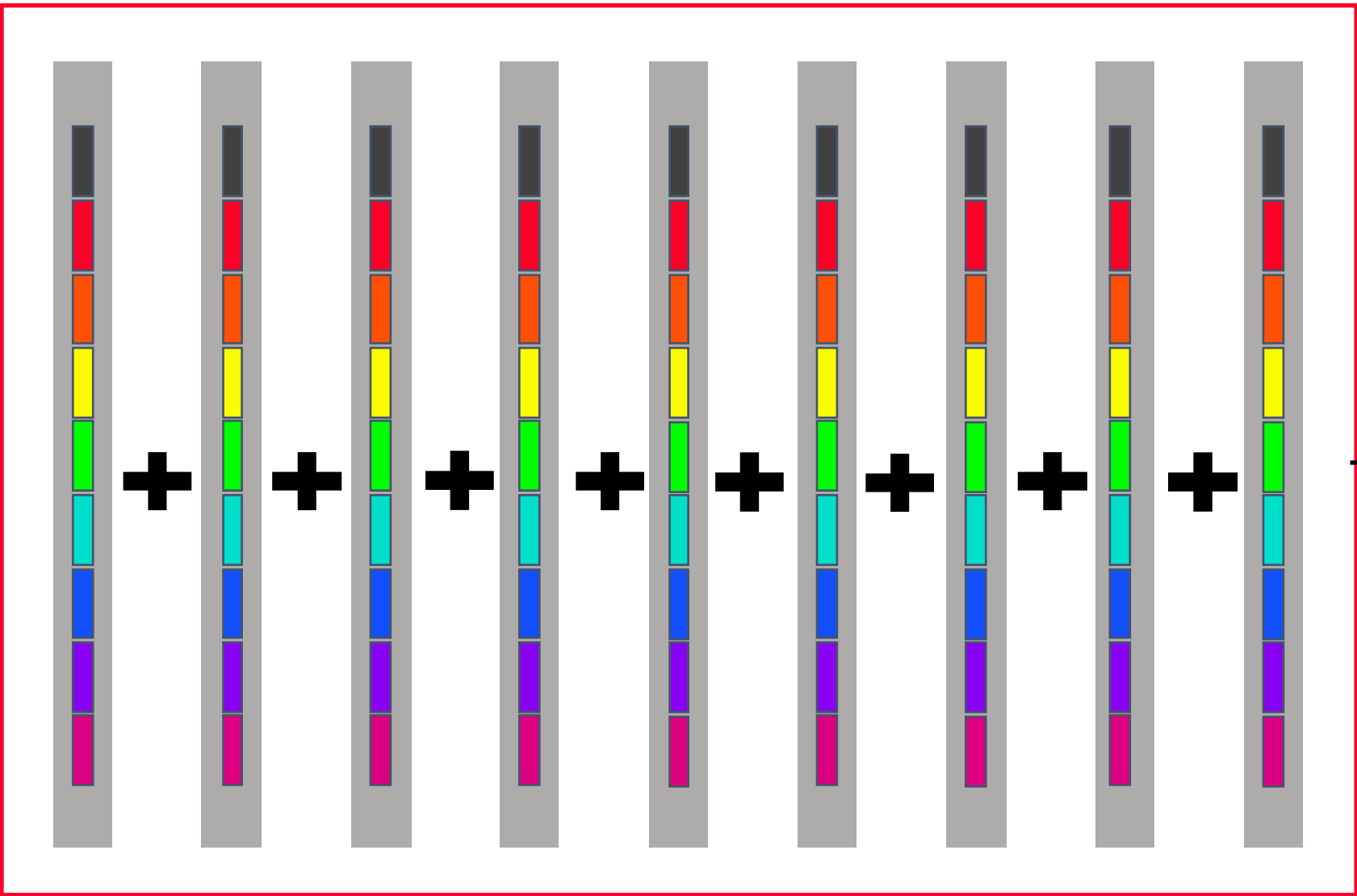


After



Reduce(-to-one)

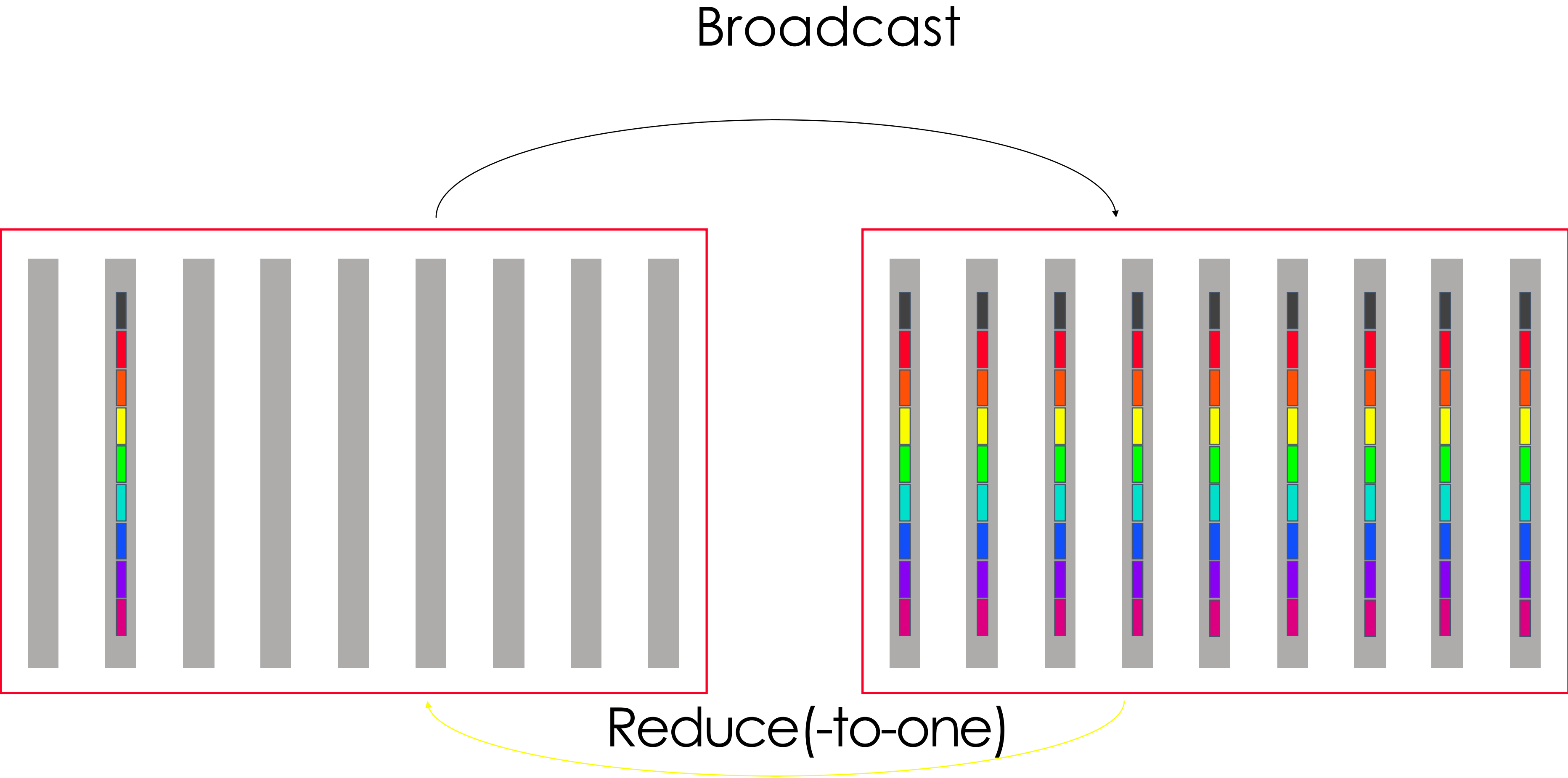
Before



After

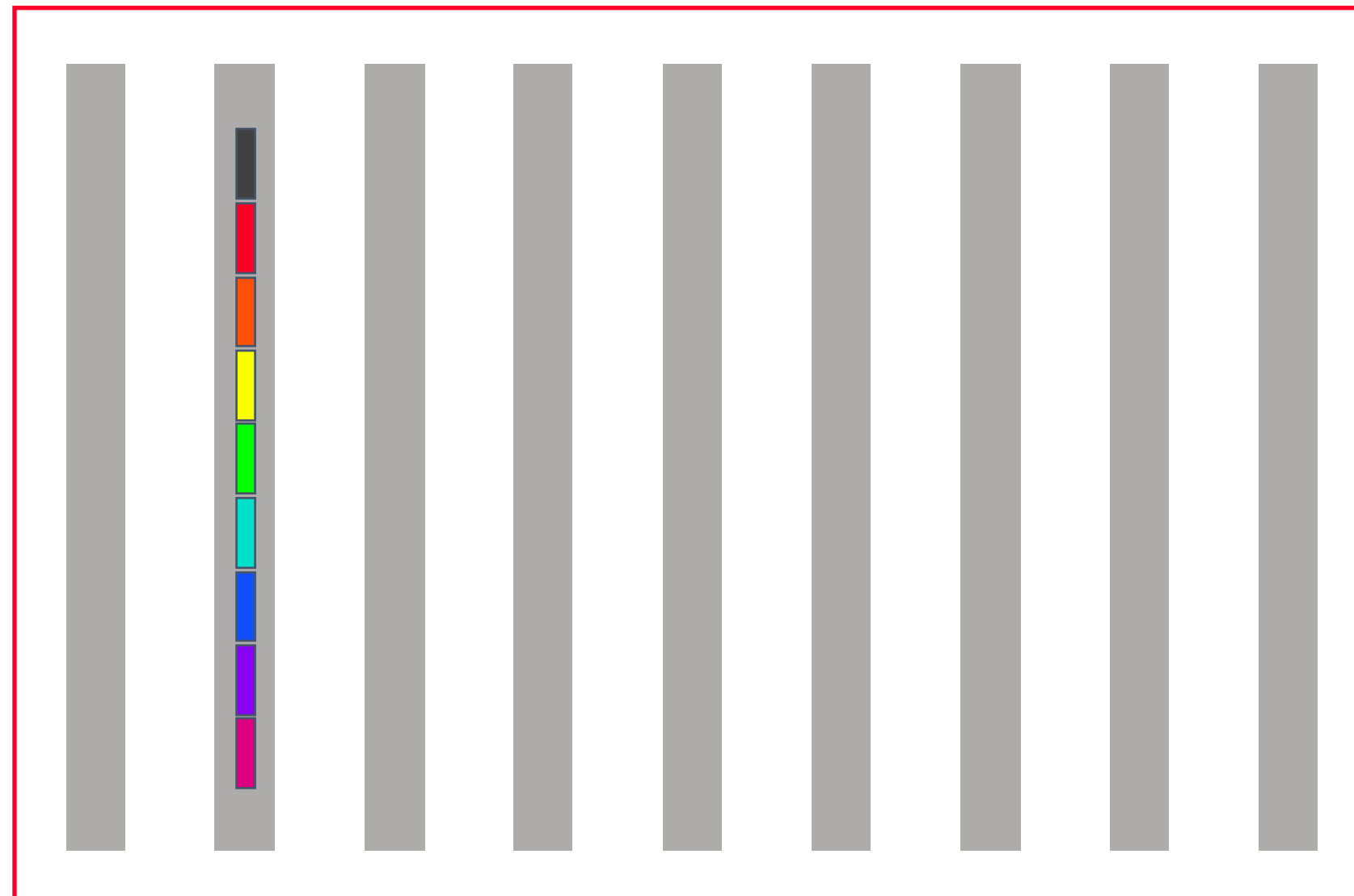


Broadcast/Reduce(-to-one)

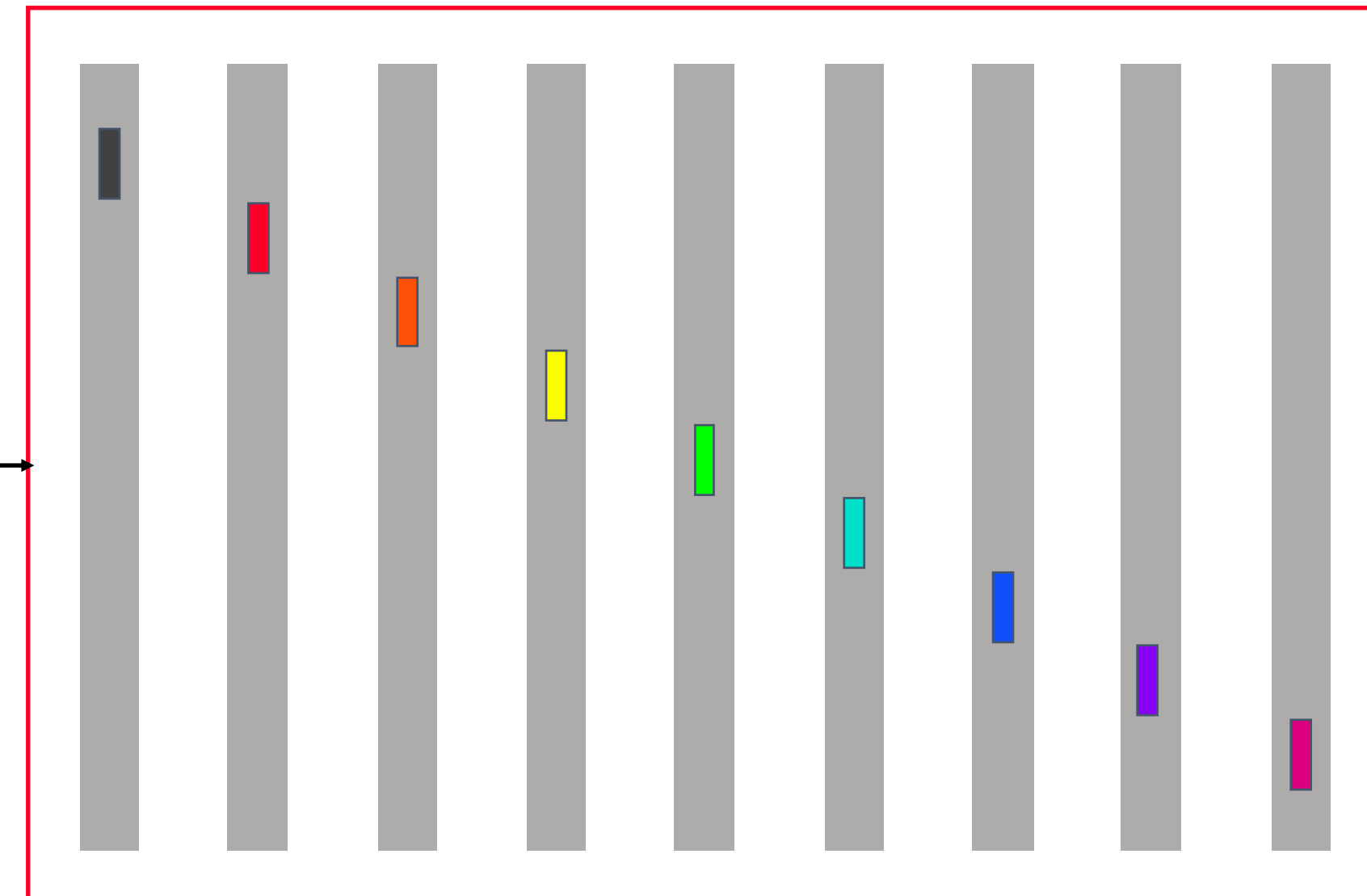


Scatter

Before

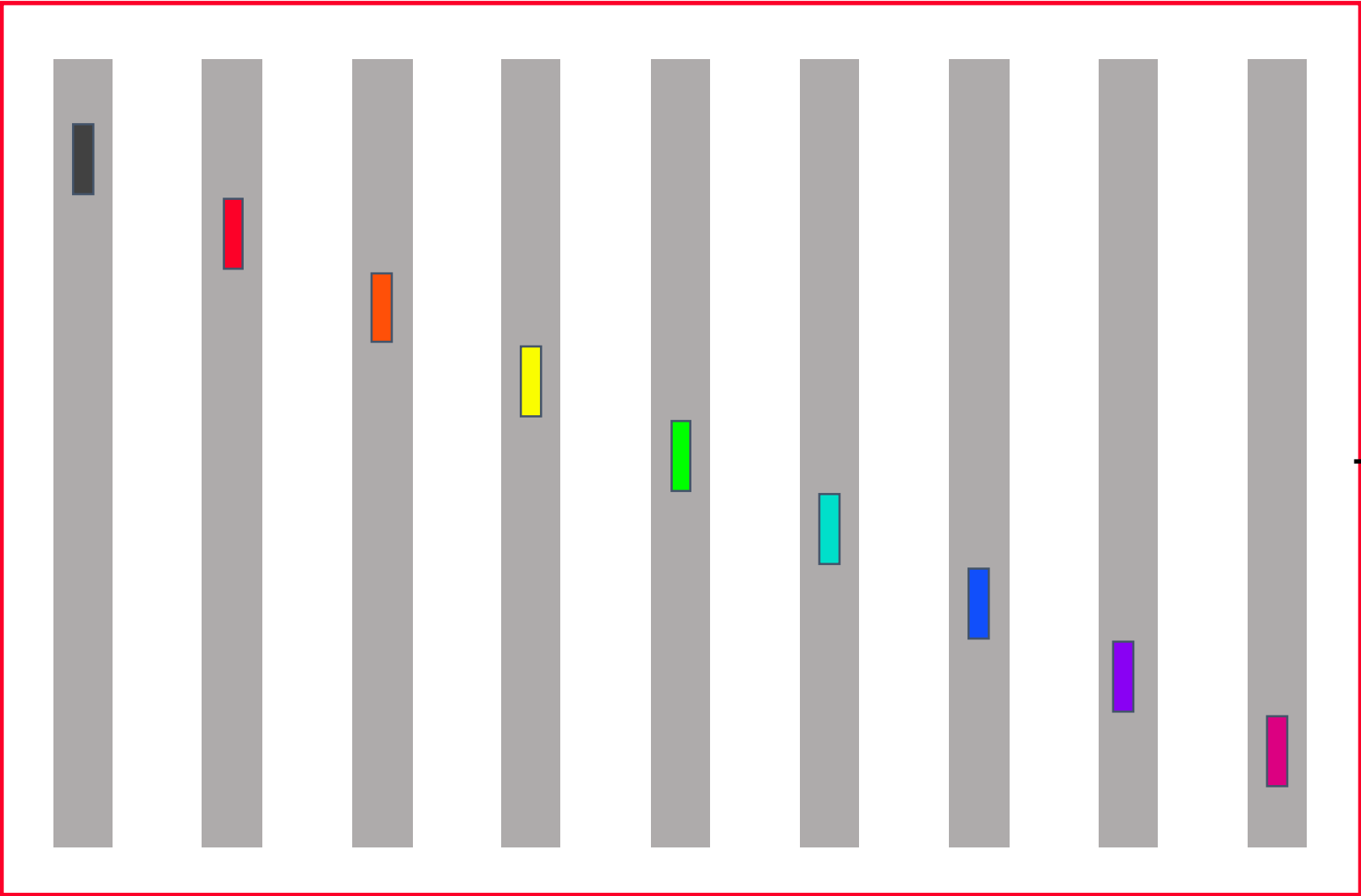


After



Gather

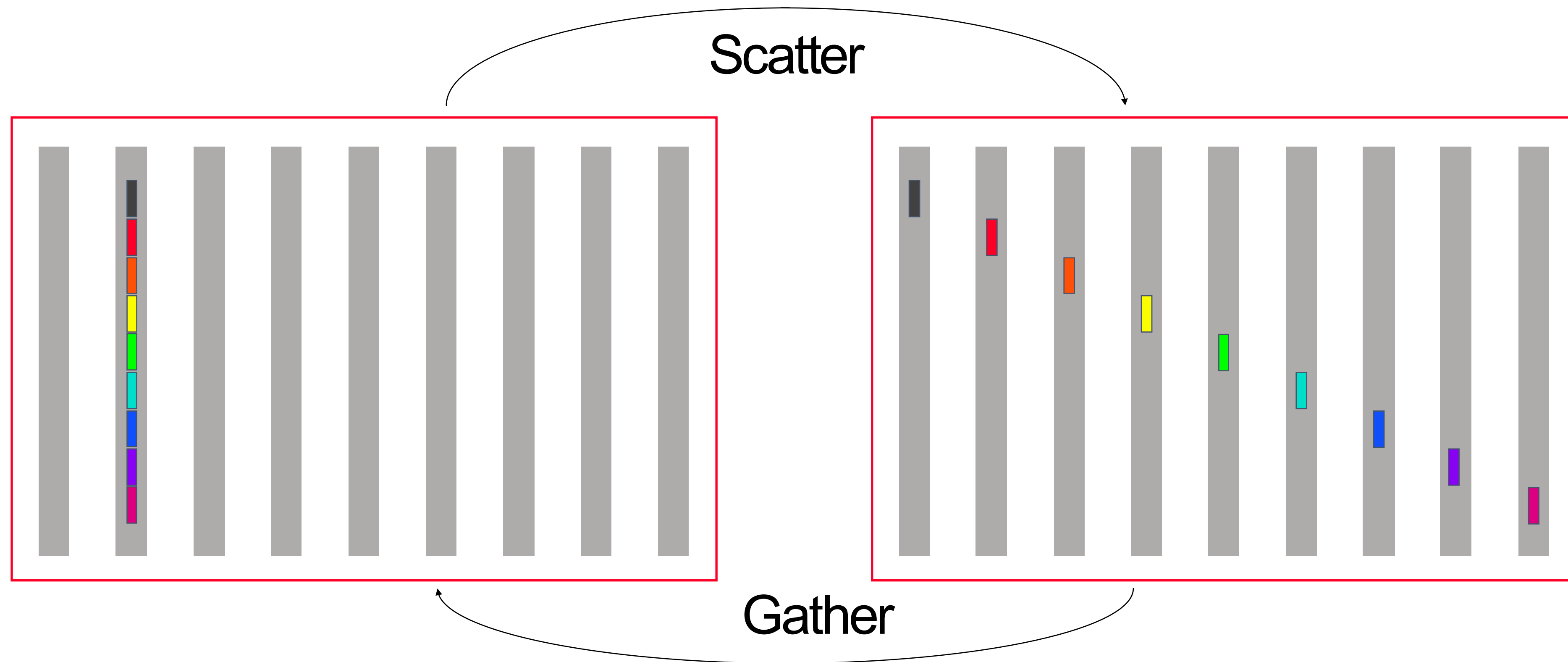
Before



After

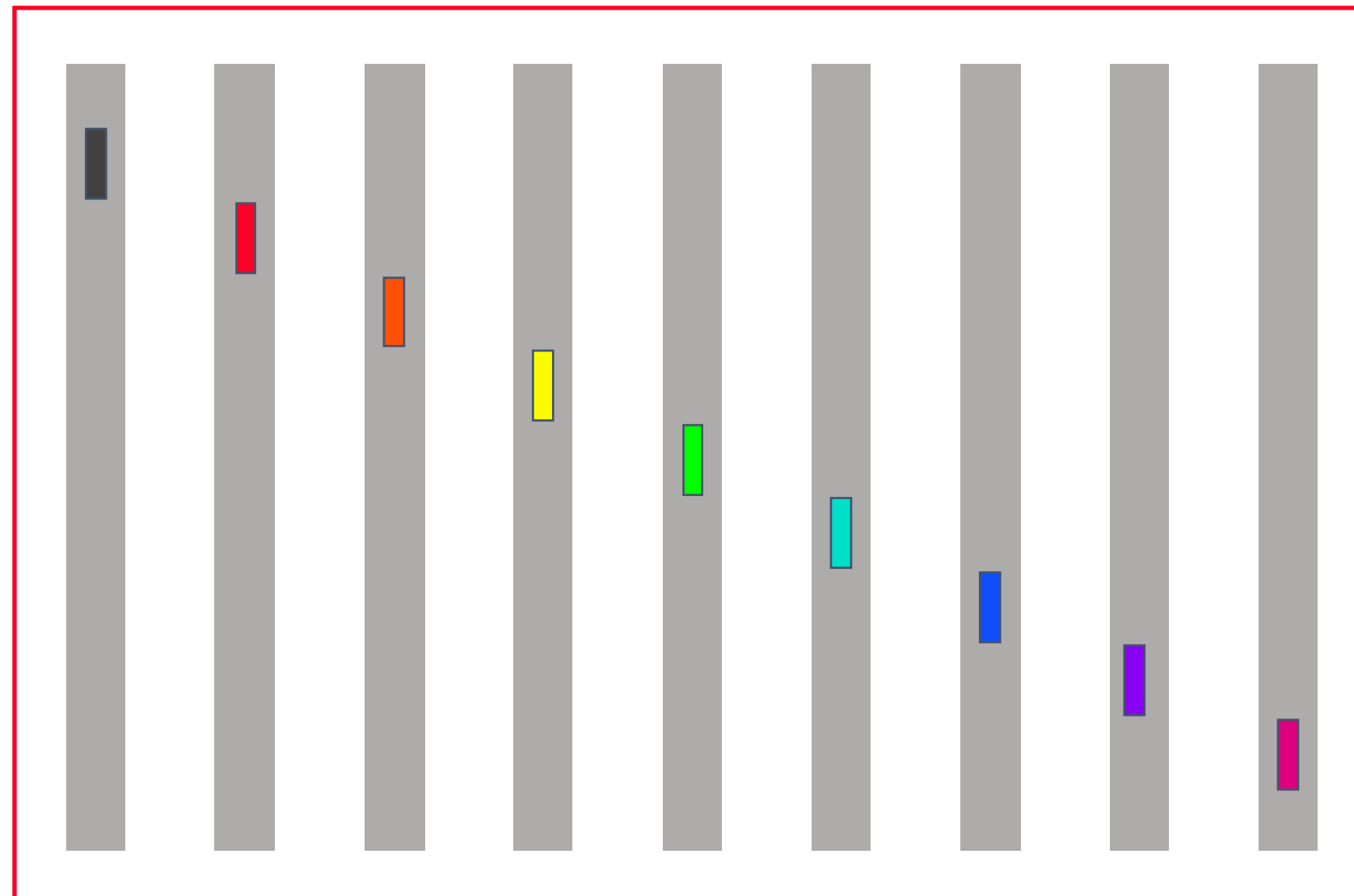


Scatter/Gather

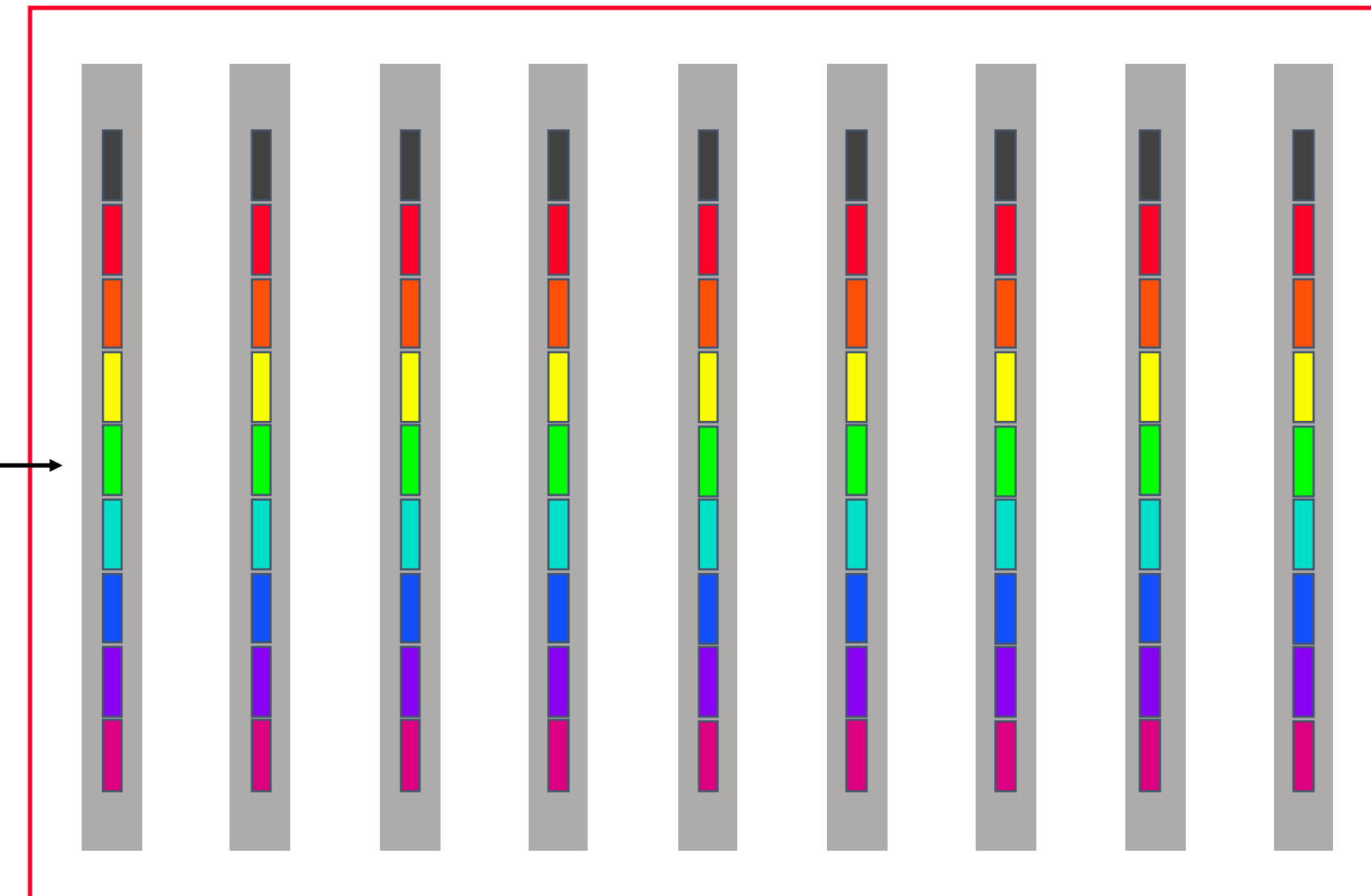


Allgather

Before

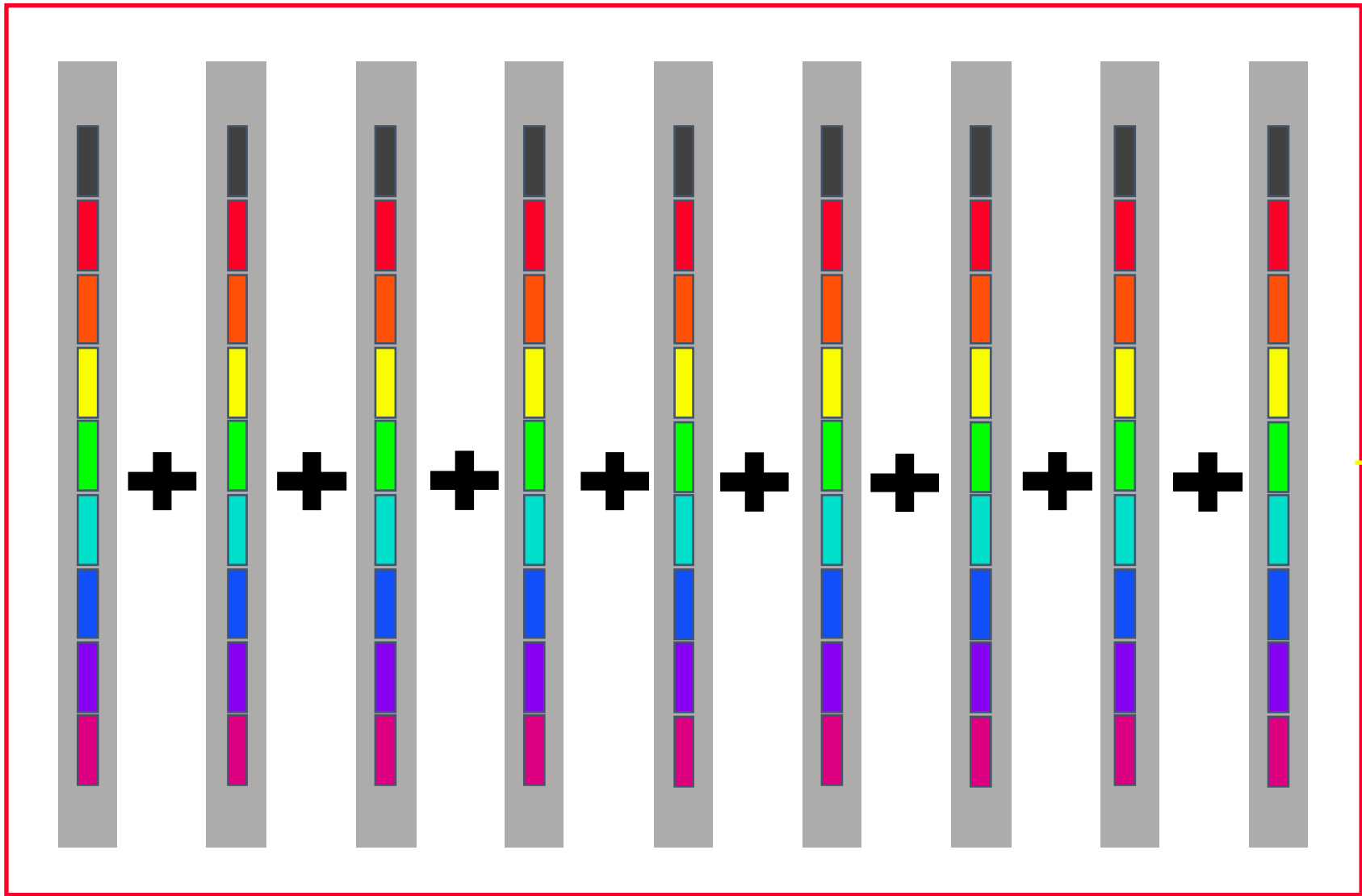


After

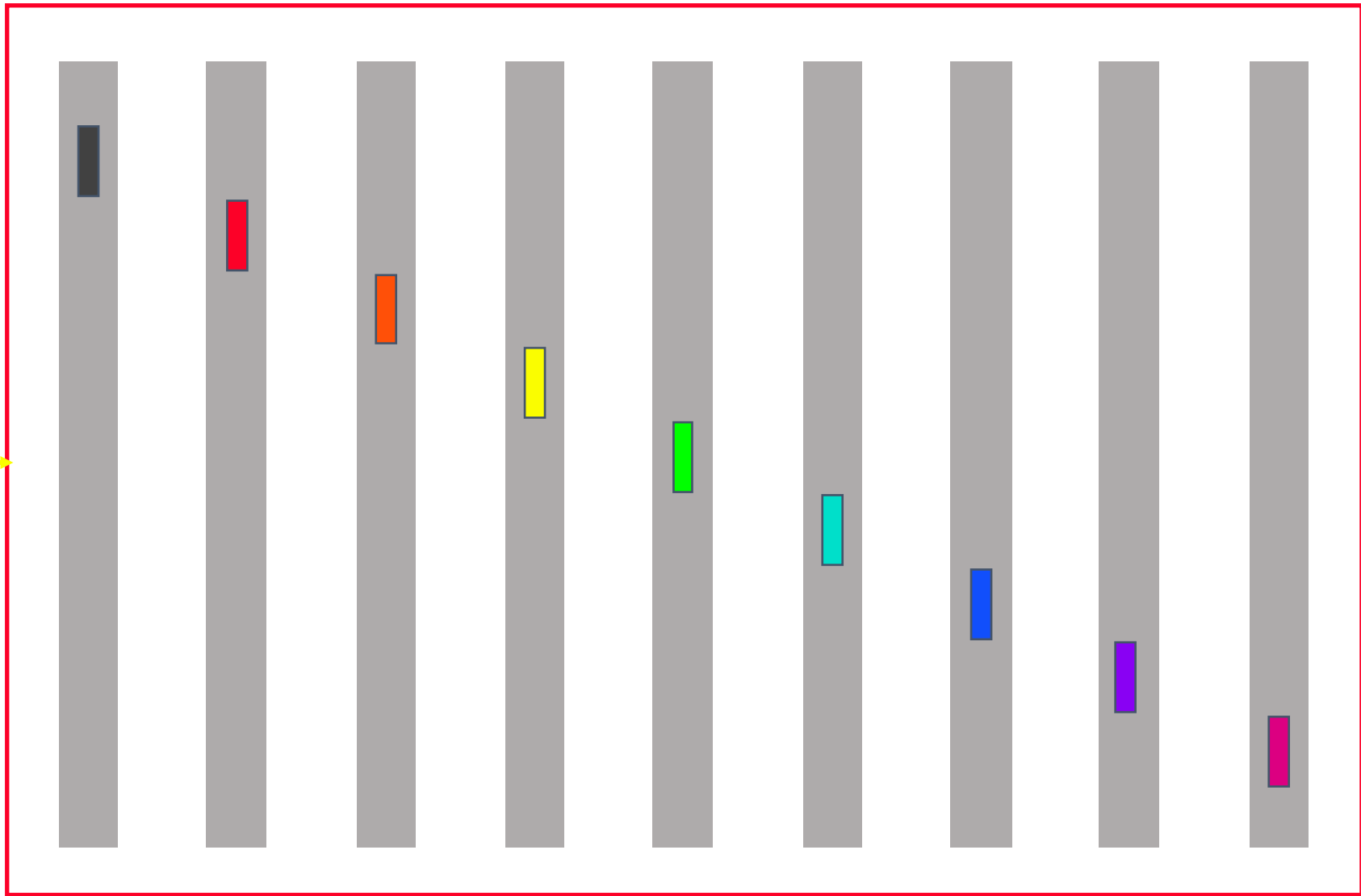


Reduce-scatter

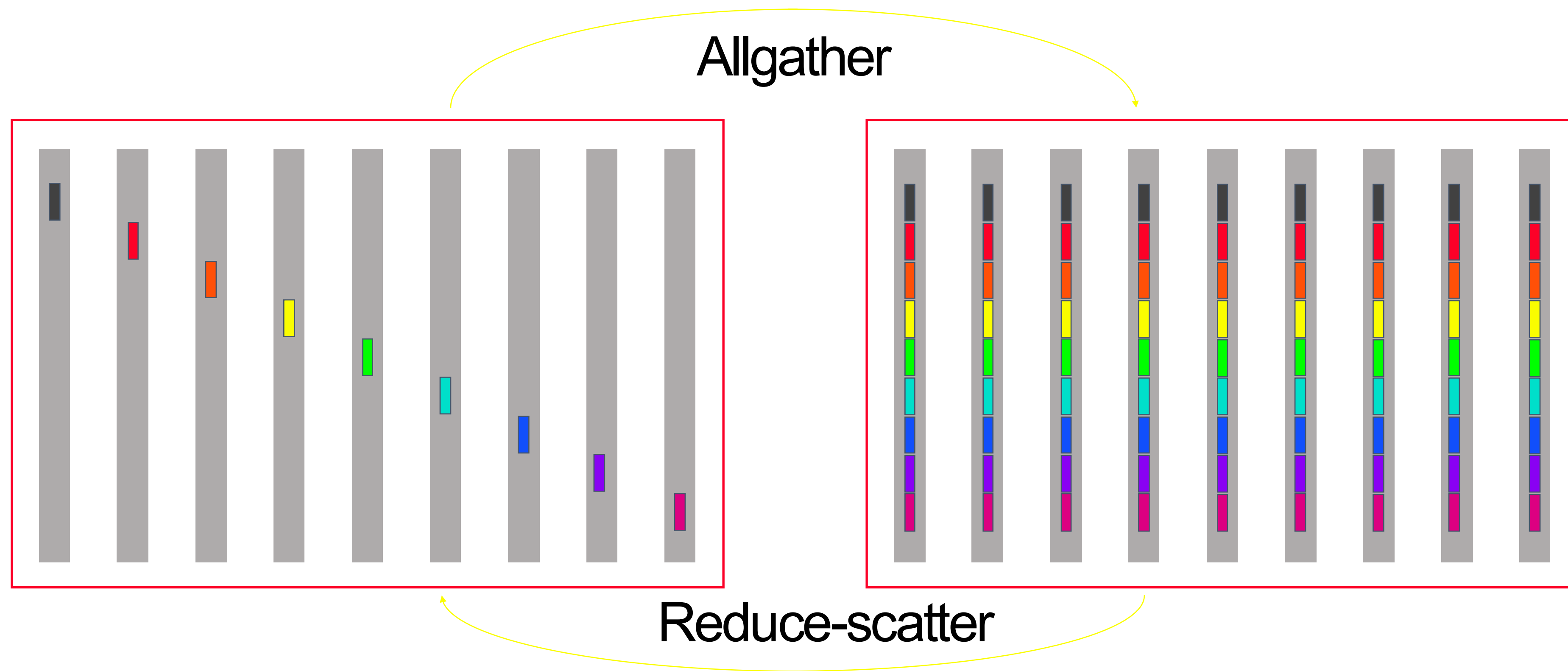
Before



After

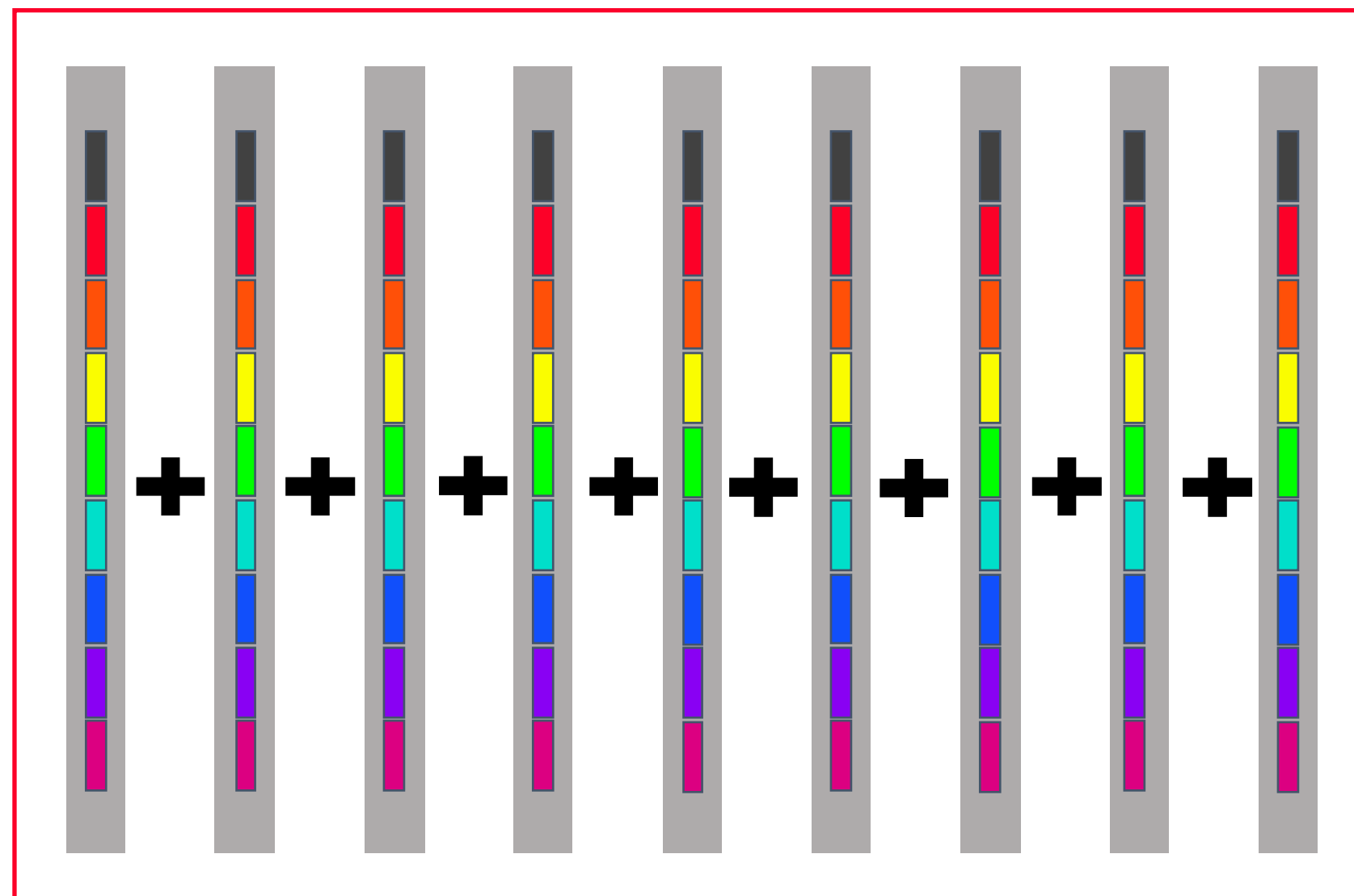


Allgather/Reduce-scatter

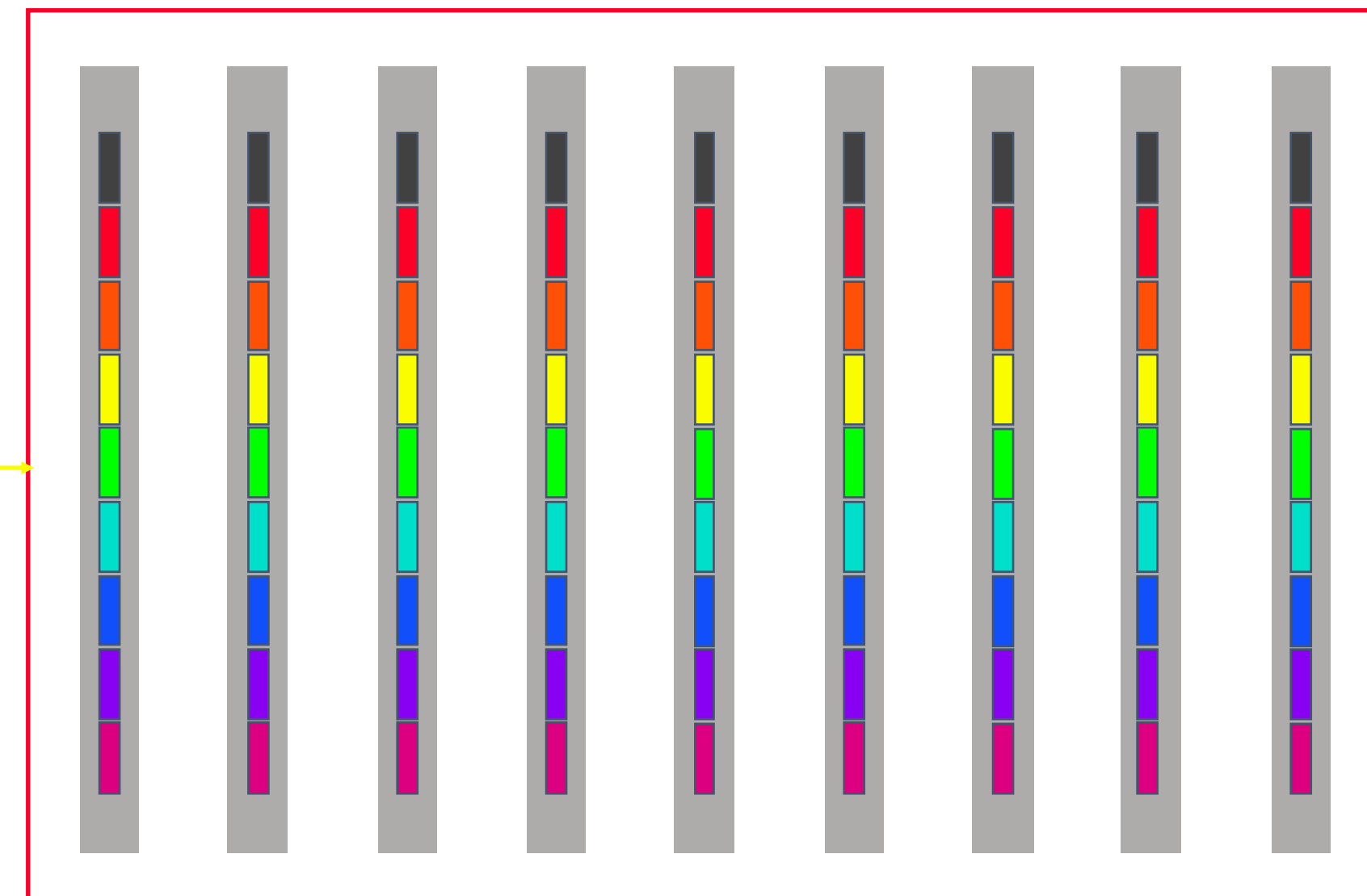


Allreduce

Before



After



Some Facts

- Collective is much more expensive than P2P
 - Collective can be assembled using many P2P
- Collective is highly optimized in the past 20 years
 - Look out for “X”CCL libraries
 - NCCL, MCCL, OneCCL
- Collective is not fault-tolerant